A Designer Tools Lab Report

on

DESIGN AND IMPLEMENTATION OF FUNCTIONAL BLOCKS IN A 64-BIT RISC-V PROCESSOR.

Submitted in partial fulfillment of the requirement for the award of degree

of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

by

KONDARAPU RESHMA - 22FE1A0472

Under the esteemed guidance of

DR. RAGHUNATHA BABU, M.Tech, (Ph.D)

Associate Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)
Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A+'** and **NBA** | **ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

2022-2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)
Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A+'** and **NBA** | **ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

CERTIFICATE

This is to certify that Desgner Tools Lab work entitled "**DESIGN AND IMPLEMENTATION OF FUNCTIONAL BLOCKS IN A 64-BIT RISC-V PROCESSOR"** is a bonafide work done by **KONDARAPU RESHMA (22FE1A0472),** under my guidance and submitted in partial fulfillment of requirements for the award of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering** by **Jawaharlal Nehru Technological University, Kakinada.**

Project Guide
Dr. RAGHUNATHA BABU, M.Tech, (Ph.D)
Associate Professor

Head of the Department
Dr. B. HARISH, M. Tech, (Ph.D)
Professor

External Examiner

ACKNOWLEDGEMENT

We are most thankful to our parents who stood as pillars of motivation and for the way they influenced and molded our lives.

We are more thankful to our chairman **Dr. LAVU RATHAIAH** who helped us to have a technical incubation by providing the required infrastructure.

We are very much thankful to our principal **Dr. PHANEENDRA KUMAR** who extended a timely help at each and every step of our academic career.

We are very thankful to our Head of Department **Dr. B. HARISH,** an amicable person who supported us very much and helped to a maximum extent and made this project successful.

We are very thankful to our beloved coordinators **Dr. K.V.VENKATA KUMAR, Ms. A.VASAVI,** and **Dr. B. BHARATHI DEVI** for inspiring all the way and arranging all the facilities and resources needed for the project. Their efforts in this aspect are beyond the preview of the acknowledgement.

We are heartly thankful to our project guide **Dr. Dr. RAGHUNATHA BABU,** M.Tech, (Ph.D)**, Associate Professor,** the person with vibrant knowledge and amicable by nature who laid a best guidelines and for the efforts made by him to make the project a successful one.

Finally, we are thankful to each and every faculty members both technical and non- technical, friends and all the people who helped us directly or indirectly in making our project a successful one.

DECLARATION

I hereby declare that the work described in this Designer Tools Lab work, entitled "**DESIGN OF REGISTER FILE UNIT, ALU, AND MEMORY UNIT USING VERILOG FOR A 64-BIT RISC-V PROCESSOR** " has been undertaken by me and this work has been submitted to VIGNAN'S LARA INSTITUTE OF TECHNOLOGY AND SCIENCE affiliated to **JNTUK, Kakinada** in the partial fulfilment of the requirements for the award of **BACHELOR OF TECHNOLOGY (B.Tech)** in the department of **Electronics and Communication Engineering,** is the result of the work done by myself under the guidance of **Dr. B. HARISH,** Professor of ECE department.

Project Associate

KONDARAPU RESHMA

(22FE1A0472)

Place: Vadlamudi

Date:

INDEX

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

This project presents the design and implementation of key functional units of a 64-bit RISC-V processor using Verilog HDL. The processor is developed based on the RV64I instruction set architecture, extending the capabilities of a 32-bit core to support 64-bit address and data operations. The design emphasizes modularity and scalability, enabling efficient execution of arithmetic, logical, and memory-related operations within a single-cycle architecture. Major components implemented include the instruction fetch unit, decode unit, arithmetic and logic unit (ALU), register file, data memory, and control unit. Each module is designed and verified through simulation to ensure correct functional behavior and adherence to RISC-V ISA specifications. The processor demonstrates effective instruction execution with accurate control signal generation and synchronized data flow between the modules. Simulation results validate the correct functionality of all functional units, achieving efficient processing at the specified clock frequency. This work establishes a foundation for further development toward a complete 64-bit RISC-V processor core, with future enhancements including pipelining, hazard detection, and performance optimization techniques such as branch prediction and cache integration.

CHAPTER 1

INTRODUCTION

RISC-V is a **free and open-standard instruction set architecture (ISA)** based on the principles of Reduced Instruction Set Computer (RISC). It is an open architecture, meaning its specifications are publicly available and can be used to design, manufacture, and sell processors and software without paying royalties or licensing fees.

1.1 History:

RISC-V began in **2010** as an academic project at the **University of California, Berkeley**, spearheaded by Professor Krste Asanović and graduate students Andrew Waterman and Yunsup Lee, with key input from computer pioneer David Patterson. The "V" signifies its status as the fifth generation of RISC (Reduced Instruction Set Computer) architecture from the Berkeley labs. The creators' primary motivation was to develop a free and open-standard Instruction Set Architecture (ISA) that was unburdened by proprietary licensing fees and legacy design constraints, making it simple, modular, and scalable. To shepherd its development and ensure its perpetual openness, the RISC-V Foundation was formed in 2015 as a non-profit organization. This commitment to openness has propelled its adoption from research and embedded systems to high-performance computing, making it a disruptive force in the semiconductor industry today.

1.2 Basic Implementation:

The basic implementation of a RISC-V processor without pipelining is known as a **Single-Cycle Processor**. In this design, every instruction completes its execution in exactly one clock cycle. This simplifies the control logic but means the clock cycle time must be long enough to accommodate the slowest instruction (usually a memory operation like Load). RISC architecture has a smaller, simpler set of instructions when compared to CISC, like x86. The key idea is that each instruction is designed to perform a single, simple task and execute in a single clock cycle. Complex operations are achieved by combining multiple simple instructions. It is a load-store architecture, which means only the processor can access external memory using dedicated instructions. All other operations, such as arithmetic and logical functions, are performed on the data i.e, already in the processor's internal registers.

1.3 Key Design Elements:

- Registers:

In a RISC-V processor, **registers** play a central role in storing and manipulating data during program execution. RISC-V defines a set of 32 general-purpose registers **(x0–x31)**, each 32-bit wide in RV32, 64-bit wide in RV64, and 128-bit wide in RV128. These registers are used to hold operands for the ALU, store intermediate values, and maintain addresses for memory operations. Importantly, register x0 is hardwired to zero, meaning it always returns the value 0, regardless of writes. The other registers have conventional uses, such as the stack pointer (x2), return address (x1), frame pointer (x8), and temporary registers for computations. This register-based design reduces memory access, making instruction execution faster and more efficient. Alongside the integer registers, RISC-V also provides floating-point registers (f0–f31) in extensions for floating-point operations

- Instruction Format:

In the RISC-V architecture, instructions follow a fixed 32-bit length format for base instructions, though there are also 16-bit compressed and 64/128-bit extended instructions in advanced versions. To maintain simplicity and regularity, RISC-V uses a small set of well-defined instruction formats, each serving a specific purpose. The R-type format is used for register-to-register operations such as arithmetic and logic instructions, where fields specify two source registers, a destination register, a function code, and an opcode. The I-type format supports immediate values and is commonly used for arithmetic with constants, load instructions, and some system calls. The S-type format is used for store instructions, encoding immediate values in a split manner to specify memory addresses. The B-type format is designed for conditional branches, holding branch offsets to alter control flow. The U-type format provides a large immediate value (upper 20 bits), suitable for instructions like LUI (Load Upper Immediate) and AUIPC (Add Upper Immediate to PC). Finally, the J-type format is used for jump instructions, carrying a larger offset for altering program execution. Together, these formats ensure that RISC-V instructions are regular, efficient, and easily decoded by hardware, supporting both simplicity in design and flexibility in operation.

- Load-Store Architecture:

RISC-V strictly adheres to the load-store model, which simplifies instruction decoding and favors fast pipelining.

1.4 Key Features:

- Open Standard and Royalty Free:

The RISC-V ISA specification is completely open and free to use by anyone. There are no licensing fees, royalties, or non-disclosure agreements required to design, manufacture, or sell a RISC-V processor. This encourages widespread adoption, innovation, and competition across the industry.

- Modularity and Extensibility:

This is the core design philosophy, allowing RISC-V to scale from small microcontrollers to large supercomputers. It supports additions of extra instructions such as: M(integer multiply/divide); A(atomic instructions for multi-core synchronization); F/D(single or double precision floating point); C(compressed instructions); V(vector processing).

- RISC-V Principles:

1. Simple and fixed-length instructions.
2. Load-store architecture.
3. Large register file.
4. Hardwired zero register.
5. Clean Design.

CHAPTER 2

LITERATURE SURVEY

2.1. Foundations: The RISC-V Instruction Set Architecture (ISA):

2.1.1 The RISC Philosophy and Open Standard:

- Key Concept**:** Compare the Reduced Instruction Set Computing (RISC) principles (simple, uniform instruction format, load-store architecture) against Complex Instruction Set Computing (CISC) to justify the choice of RISC-V.
- Essential Reading**:** Patterson and Henne**ssy's** influential work on computer architecture and the original papers from UC Berkeley that introduced the RISC-V ISA (e.g., Krste Asanovic et al.). These papers emphasize the advantages of a clean-slate, open-source, and royalty-free ISA.

2.1.2 Processor Microarchitecture Design:

Functional Block	Key Literature Focus
Instruction Fetch (IF)	Program Counter (PC) design, Instruction Memory interface, handling of compressed instructions (C extension, if included).
Instruction Decode (ID)	Opcode decoding logic, **Register File** structure (32x64-bit registers for RV64), and immediate generation logic (handling U-type, I-type, S-type, etc., formats).
Arithmetic Logic Unit (ALU)	Design of the 64-bit combinational ALU. Focus on arithmetic functions (e.g., ripple-carry vs. look-ahead adders for high-speed design) and logical/shift operations.
Control Unit	Design methodologies for the main controller (e.g., hardwired control vs. microprogrammed control) that generates the control signals for the entire datapath.

Table-01 Processor Micro-Architecture Design

2.1.3 Implementation and Verification Methodologies:

- Hardware Description Language (HDL):

Key Concept**:** The use of Verilog or VHDL for Register Transfer Level (RTL) design. Literature should cover best practices for writing synthesizable HDL code for an ASIC or FPGA target.

- Physical Implementation (Target Technology):

FPGA Implementation**:** Research papers on implementing RISC-V cores on Field-Programmable Gate Arrays (FPGAs) (e.g., Xilinx or Intel FPGAs). These sources will discuss resource utilization (LUTs, Flip-Flops) and timing analysis.

- Verification and Testing

Discuss the importance of a robust verification strategy. This often involves using a "Golden Model" (like the Spike RISC-V ISA simulator) to check the functional correctness of your RTL implementation. Test Benches and Suites**:** Reference the use of standard compliance test suites (e.g., the RISC-V Architectural Test Suite) and methodologies like instruction stream randomization.

2.1.4 Design Challenges and Future Work:

•Design Challenges: Papers discussing the complexities of implementing 64-bit data paths, managing exceptions and interrupts, and optimizing the core for low power consumption.
•Advanced Microarchitecture: Research on modern RISC-V designs, such as superscalar (executing multiple instructions per cycle) or out-of-order execution cores, can be included to show a path for future expansion of the project.

From the review of the various studies done by various authors, it is understood that there is a need for the betterment of performance and hence inevitable trade-offs in various performance parameters like computational speed, power consumption, and area of the processor are required. In this project, our

2.2 Conclusion For Literature Survey:

From the review of the various studies done by various authors, it is understood that there is a need for the betterment of performance and hence inevitable trade-offs in various performance parameters like computational speed, power consumption, and area of the processor are required. In this project, our main aim is to design a Register File Unit, an Arithmetic and Logical Unit, and a Memory Unit block for a 64-bit RISC–V PROCESSOR. It has become a popular option for System On Chip(SoC) and Internet of Things(IoT) technologies. Because of its open-source nature and strong safety measures, it may compete with popular ARM technology. This work covers the design of a current hardware design technique-based, compact, open-source implementation of a RISC-V CPU. Thorough testing is done, with the implementation intended for FPGA's deployment. The primary objective is to develop a RISC-V processor that is accessible to beginners and sufficiently portable to be deployed on a modest-scale FPGA. We use VERILOG to design the functional blocks for a 64-bit RISC-V PROCESSOR. Hardware Description Language offers a resource-efficient and time-saving approach. The simple instruction set provides valuable insights into the hardware requirements necessary for effective execution. Key components such as ALU, RAM, Control unit, Register file, and Instruction register are integrated within the processor design.

CHAPTER-3

PROPOSED SYSTEM.

RISC-V (pronounced "risk-five") is a free and open-standard Instruction Set Architecture (ISA**)** based on Reduced Instruction Set Computer (RISC**)** principles.
Unlike proprietary ISAs like ARM or x86, RISC-V is completely open and royalty-free, meaning anyone can use it, implement it, and extend it without paying licensing fees. Its design emphasizes simplicity, modularity, and extensibility, making it suitable for a wide range of applications, from tiny embedded systems to high-performance supercomputers.

3.1 32-Bit RISC-V:

The 32-bit RISC-V architecture is denoted as RV32I (RISC-V 32-bit Base Integer ISA). It is the foundational, minimal integer instruction set.
•Register Width: All general-purpose registers (x0 to x31) are 32 bits wide.
•Address Space**:** The Program Counter (PC) and addresses are 32 bits wide, which limits the maximum addressable memory to 232 bytes, or 4 GB.
•Target Application**:** RV32I is primarily used for embedded systems, microcontrollers (MCUs), and low-power applications where memory requirements are small, and cost/energy efficiency is paramount.

3.2 Need for 64-Bit RISC-V:

The 64-bit RISC-V architecture (RV64I) is the clear preference for general-purpose computing, high-performance computing (HPC), and any system intended to run a modern operating system (OS) like Linux, due to its enhanced capabilities in memory management and data processing.

- Massive Increase in Addressable Memory:

32-bit Limit**:** Limited to 4 GB of addressable memory (232 bytes). This is insufficient for servers, workstations, and even many modern desktop applications.
64-bit Capability**:** Supports an address space of 264 GB, which theoretically allows addressing **16** Exabytes (16 Billion GB) of memory. This capacity eliminates memory constraints, which is critical for large databases, virtualization, and running data-intensive applications.

- Higher Computational Throughput:

Data Path Width**:** RV64I processors have a 64-bit data path. This means the ALU (Arithmetic Logic Unit), registers, and internal buses can process 64-bit integer values in a single clock cycle.
•Performance**:** For tasks involving high-precision arithmetic (e.g., scientific simulation, cryptography, large-number calculations), the 64-bit core provides a significant performance boost compared to a 32-bit core, which would require multiple instructions to perform a 64-bit operation.

- Compatibility with Modern Software and OS:

.
OS Support: Modern, general-purpose operating systems are almost exclusively compiled for 64-bit architectures because they require the larger address space to function efficiently and securely

3.3 Basic RISC-V Architecture:

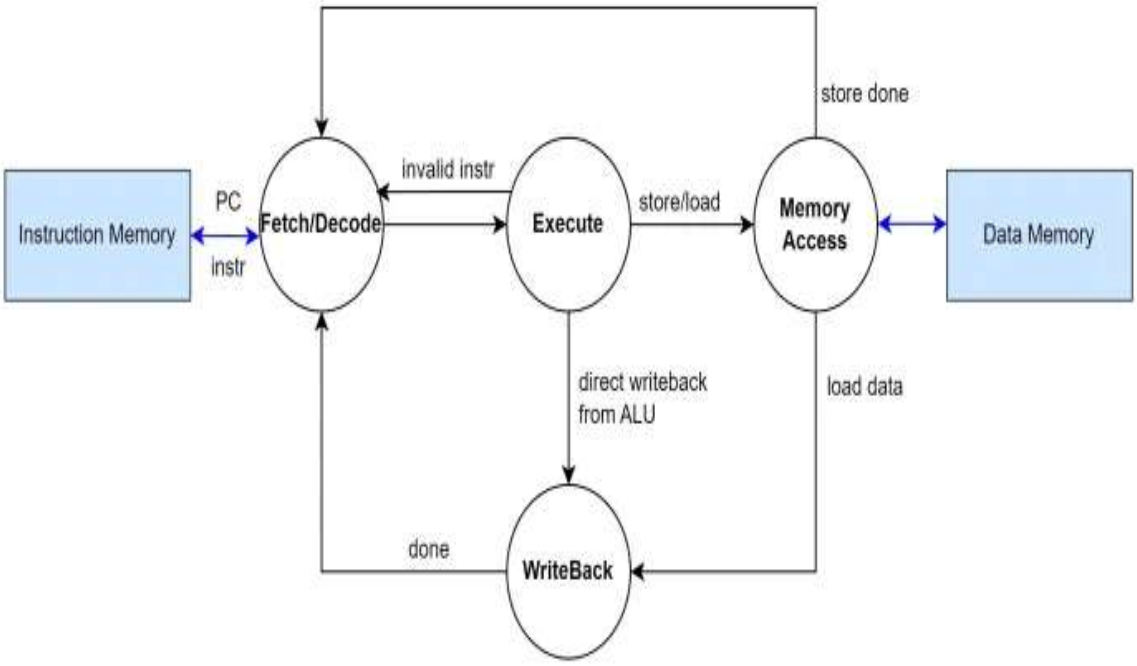

Fig.01 RISC-V Architecture

RISC-V is an open and free ISA based on the philosophy of Reduced Instruction Set Computing (RISC). The ISA uses a small, highly optimized set of instructions, each performing one basic operation. The base instructions are always 32 bits wide (even in 64-bit RISC-V, RV64I), which simplifies the fetching and decoding processes. Functional blocks in the processor are:

- Instruction Fetch Unit.
- Instruction Decode Unit.
- Register File Unit.
- Arithmetic and Logical Unit.
- Memory Unit.

Each block implementation is explained below:

3.4 Instruction Fetch Unit:

In RISC-V, the instruction set architecture(ISA) is designed with a small number of fixed instruction formats. This regularity is a key principle of simplified hardware design. There are six basic instruction types in RISC-V:

- R-Type:

Register to register and use for arithmetic and logical operations that operate on source registers and write the result to the destination register. The instruction representation format is:

funct7 rs2 rs1 funct3 rd opcode

- I-Type:

It performs immediate and load operations between a register and a 12-bit signed immediate value. Loads data from the register. The instruction representation format is:

imm[11:0] rs1 Funct3 rd opcode

- S-Type:

This operation is mostly used for "Store" operations, which write data from a register to the memory location. The instruction representation format is:

imm[11:5] rs2 rs1 funct3 imm[4:0] opcode

- B-Type:

Branch type is used for conditional branch instructions, which alter the program flow of execution based on a comparison between two registers. The instruction representation format is:

imm[12] imm[10:5] rs2 rs1 funst3 imm[4:1] imm[11] opcode

- U-Type:

Upper immediate instruction is used to load a 20-bit immediate value into the upper 20 bits of the register, with lower bits set to zero. This is often used in conjunction with an I-type instruction to construct a full 32-bit address. The instruction representation format is:

imm[31:12] rd opcode

- J-Type:

This instruction is used for conditional jumps that change the program flow to a new address. The target address is calculated relative to the program counter.The instruction representation format is:

imm[20-bit] rd opcode

Operation: This stage is responsible for determining the address of the next instruction and retrieving that instruction from the Instruction Memory. The Program Counter (PC) holds the memory address of the current instruction. Instruction Memory is the read-only component (for this stage) that outputs the 32-bit instruction corresponding to the PC address. PC Adder will calculate the address of the next instruction by simply adding 4 to the current PC (PC+4), as each instruction is 4 bytes (32 bits).

3.5 Instruction Decode Unit:

Operation: This is the "brain" stage where the instruction is analyzed and all operands are prepared for the next stage.

Instruction Decoder / Control Unit: The core logic that reads the opcode, function codes (funct3,funct7), and generates all necessary control signals for the subsequent stages.

3.6 Register File Unit:

The Register File is essentially a small, fast memory array with dedicated read and write access ports.

Port/Signal	Direction	Width (in bits)	Description
Clock (clk)	Input	1	Synchronizes the write operation.
. Write Enable (RegWrite)	Input	1	Control signal: High (1) enables the write; Low (0) holds the current values.
Read Address 1 (rs1_addr)	Input	5	Address of the first register to read (x0 - x31).
Read Address 2 (rs2_addr)	Input	5	Address of the second register to read (x0 - x31).
Write Address (rd_addr)	Input	5	Address of the register to be written to (x1 - x31).
. Write Data (rd_data)	Input	XLEN	The data to be stored in the register (ALU result or Memory data).
Read Data 1 (rs1_data)	Output	XLEN	Data read from rs1_addr.
Read Data 2 (rs2_data)	Output	XLEN	Data read from rs2_addr.

Table-02 Register File Unit

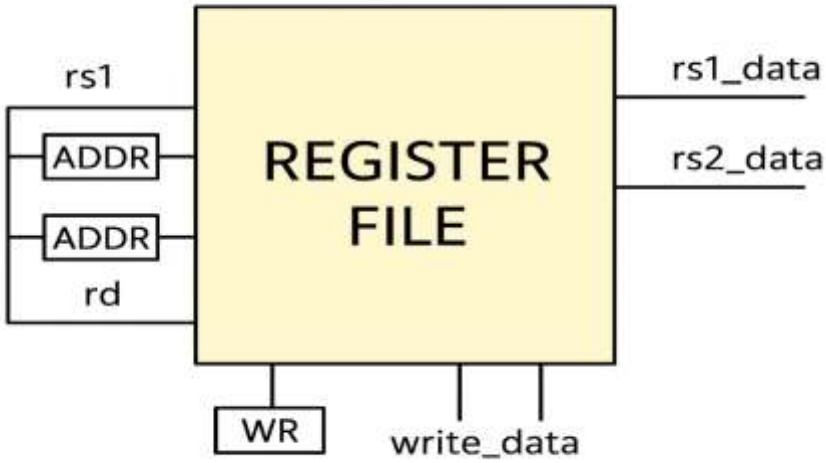

Fig.02 Register File Unit

The Register File Unit in a 64-bit RISC-V processor is one of the core components responsible for storing and providing data to the processor during instruction execution. It acts as a small, high-speed memory within the CPU that holds temporary data, operands, and intermediate results. Instead of performing operations directly on memory, the RISC-V architecture relies heavily on registers to execute instructions efficiently, making the register file a key element in achieving fast computational performance. In a 64-bit RISC-V processor, the register file consists of 32 general-purpose registers, labeled from x0 to x31, where each register is 64 bits wide. This means that every register can hold 64-bit data values, enabling the processor to handle large integers, memory addresses, and high-precision computations efficiently. The register file typically has two read ports and one write port, allowing the processor to read two source registers and write the result back to one destination register in a single clock cycle. The register x0 is a special register that is hardwired to zero and cannot be modified; it is often used to simplify instruction encoding and immediate operations.. During operation, when an instruction is fetched, it contains the addresses of the source and destination registers. For example, in the instruction ADD x5, x2, x3, the register file reads the contents of registers x2 and x3 using the two read ports, sends them to the Arithmetic Logic Unit (ALU), and after computation, the result is written back into register x5 using the write port. The read decoders in the register file select which registers to read, while the write decoder determines which register to update. The write enable signal ensures that writing happens only when required and prevents accidental modification of registers, especially x0. The internal structure of the register file can be visualized as a 32 × 64-bit memory array controlled by decoders and multiplexers. Two read decoders select the appropriate registers for reading, and one write decoder activates the register to be written. The outputs are connected to the ALU, while the inputs come from the ALU's result or other data sources during the write-back stage. This design allows for simultaneous reading of two operands and the writing of one result, ensuring smooth data flow through the pipeline stages of the processor.

When compared to a **32-bit register file**, the 64-bit version offers several advantages. A register file can store only 4 bytes per register, while a 64-bit register file can store 8 bytes, effectively doubling the data capacity per register. This increase in data width enables the processor to perform 64-bit arithmetic operations and access a much larger memory address space — up to 16 exabytes, compared to the 4 GB limit of a 32-bit system. This makes the 64-bit register file more suitable for modern applications that require large memory handling, high-speed data processing, and complex computations such as encryption, multimedia processing, and scientific calculations.

3.7 Arithmetic and Logical Unit:

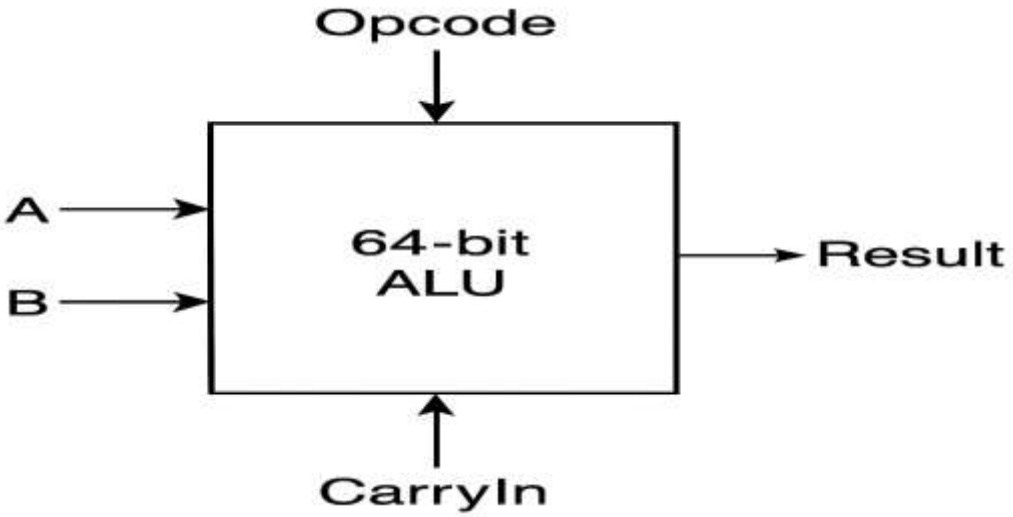

Fig.03 ALU

The Arithmetic Logic Unit (ALU) in a 64-bit RISC-V processor is the core component responsible for performing all arithmetic (like addition, subtraction, multiplication, division) and logical (like AND, OR, XOR, NOT, shift, and comparison) operations. It takes two 64-bit input operands—usually read from the register file—and produces a 64-bit output result. The ALU operation to be performed is controlled by an ALU control signal generated from the instruction's opcode and function fields during the decode stage. During execution, when an instruction like ADD x3, x1, x2 is encountered, the control unit decodes it and sends signals to the ALU to perform addition. The ALU takes the 64-bit values stored in registers x1 and x2, adds them, and stores the 64-bit result in register **x3**. Similarly, for a logical operation such as AND x4, x5, x6, the ALU performs a bitwise AND operation on the 64-bit inputs from x5 and x6 and stores the result in x4. The zero flag or less-than flag may also be set based on the result for conditional branching.

A 64-bit ALU can process larger data and address ranges in a single clock cycle compared to a 32-bit ALU. This means it can handle numbers up to 2^{64}instead of 2^{32} ,allowing more precise and efficient computation—especially important in modern applications like encryption, scientific computation, multimedia processing, and operating systems that manage large memory spaces. It also supports 64-bit addressing, enabling direct access to vastly larger memory (over 4 GB) without complex segmentation. Overall, a 64-bit ALU provides higher performance**,** scalability, and efficiency, making it ideal for modern high-performance computing systems.

3.8 Memory Unit:

The memory unit in a 64-bit RISC-V processor is responsible for storing and retrieving data and instructions during program execution. It acts as an interface between the **processor** and the main memory (RAM**)**. The memory unit handles load and store operations, where *load* instructions fetch data from memory into registers, and *store* instructions write data from registers back into memory. In a 64-bit RISC-V processor, the memory unit works with 64-bit addresses and 64-bit data words, meaning it can directly access and process larger chunks of data per instruction. When a load instruction (e.g., LD x5, 0(x10)) is executed, the memory unit takes the address from register x10, fetches 64 bits (8 bytes) of data from that memory location, and places it into register x5. Similarly, for a store instruction (e.g., SD x6, 8(x10)), it stores the 64-bit content of register x6 into the memory location pointed to by x10 + 8. The memory unit includes address generation logic, data alignment circuits, and control signals for read/write operations, ensuring correct data flow between CPU and memory.

Suppose the processor wants to load a 64-bit integer stored at memory address 0x1000 into register x2.

- The instruction LD x2, 0(x1) is executed, where x1 contains 0x1000.
- The memory unit calculates the effective address (0x1000 + 0 = 0x1000), fetches 8 bytes (64 bits) of data from that address, and loads it into x2.

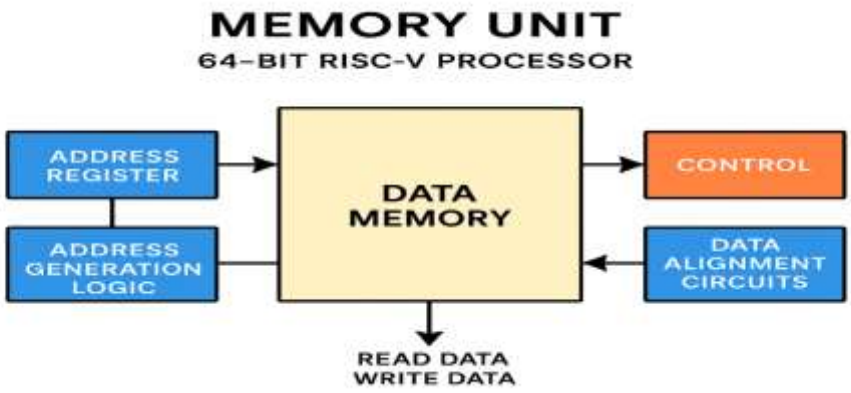

Fig.04 Memory Unit

A 64-bit RISC-V processor's memory unit is more efficient and powerful than that of a 32-bit system because it can address a much larger memory space (2^{64} bytes vs. 2^{32} bytes) and process 64-bit data in a single operation. This wider data path increases computational performance, especially in applications involving large datasets, encryption, scientific computation, and multimedia processing. It also allows the processor to handle larger integers and pointers directly, reducing the number of instructions needed for complex calculations. In short, the 64-bit memory unit improves speed, efficiency, and scalability compared to the 32-bit version.

3.9 Working of 64-bit RISC-V Processor:

The **64-bit RISC-V processor** operates through a sequence of coordinated functional blocks that execute each instruction in a pipelined manner. The operation begins with the **Instruction Fetch (IF) unit**, where the processor uses the Program Counter (PC) to fetch the next 32-bit instruction from instruction memory. The PC is then updated by adding 4 to point to the next instruction, or replaced by a branch or jump target address if control flow changes occur. The fetched instruction and PC are passed to the next stage, the **Instruction Decode (ID) unit**, which interprets the instruction fields such as opcode, function bits, and register identifiers (rs1, rs2, rd). In this stage, the **Register File (RF)** reads the source operands (rs1 and rs2) from its 32 registers, each 64 bits wide in a 64-bit processor, while the decode logic also extracts and sign-extends the immediate value based on the instruction format (R, I, S, B, U, or J type). The **Control Unit** generates appropriate control signals such as ALU operation type, memory access permissions, and writeback selection, which guide the subsequent stages.

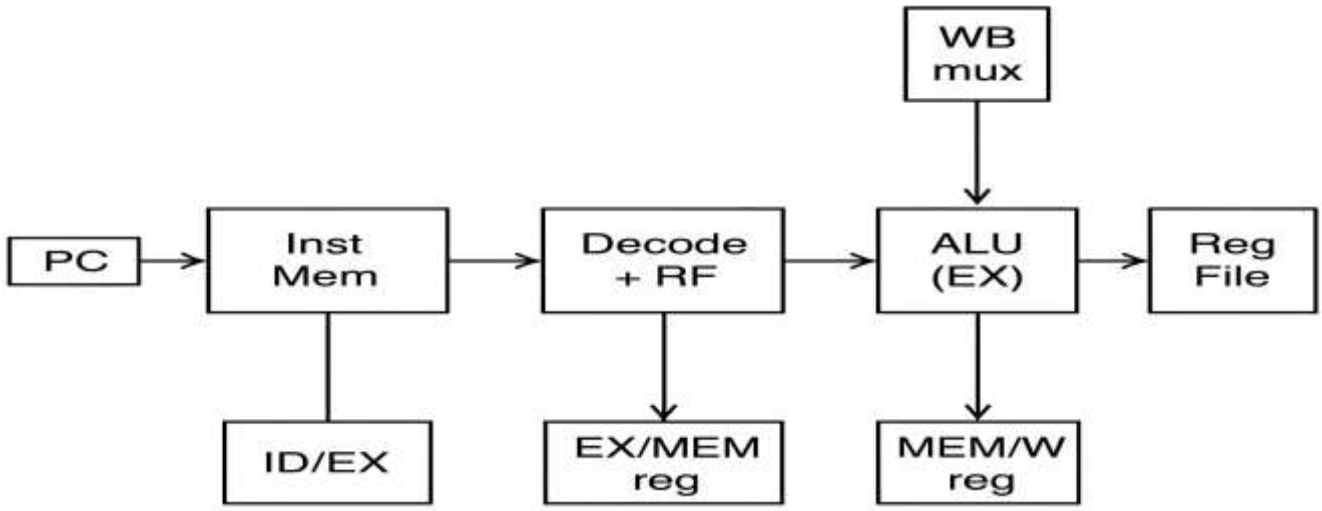

Fig.05 Block Diagram for RISC-V working

The execution then moves to the Execute (EX) stage, where the Arithmetic Logic Unit (ALU) performs the required 64-bit operation such as addition, subtraction, bitwise logic, shifting, or comparison. For branch instructions, the ALU also compares operands and determines whether a branch is taken, computing the target address accordingly. The result from the ALU and the control signals are then passed to the Memory (MEM) stage, which handles data transfer instructions. If the instruction is a load, the data memory is accessed using the computed address, and the fetched data is sign- or zero-extended to 64 bits before being forwarded. If it is a store instruction, the value from the source register is written into the memory at the specified address. For arithmetic and logical operations that do not require memory, this stage simply passes the ALU result forward.

Finally, in the Writeback (WB) stage, the result from either the ALU or memory is written back into the destination register (rd) in the register file if the control signal RegWrite is enabled. This completes the instruction's lifecycle. To enhance performance, the 64-bit RISC-V processor commonly employs pipelining, allowing multiple instructions to be processed concurrently—each instruction at a different stage of execution. However, pipelining introduces hazards such as data, control, and structural hazards, which are mitigated through techniques like data forwarding, stalling, and branch prediction. Compared to a 32-bit processor, a 64-bit RISC-V system can handle larger data values and address a significantly larger memory space (2^{64} bytes). This wider datapath makes it more suitable for high-performance computing, data-intensive applications, and systems that require large addressable memory, while retaining the modularity, open-source flexibility, and simplicity that define the RISC-V architecture.

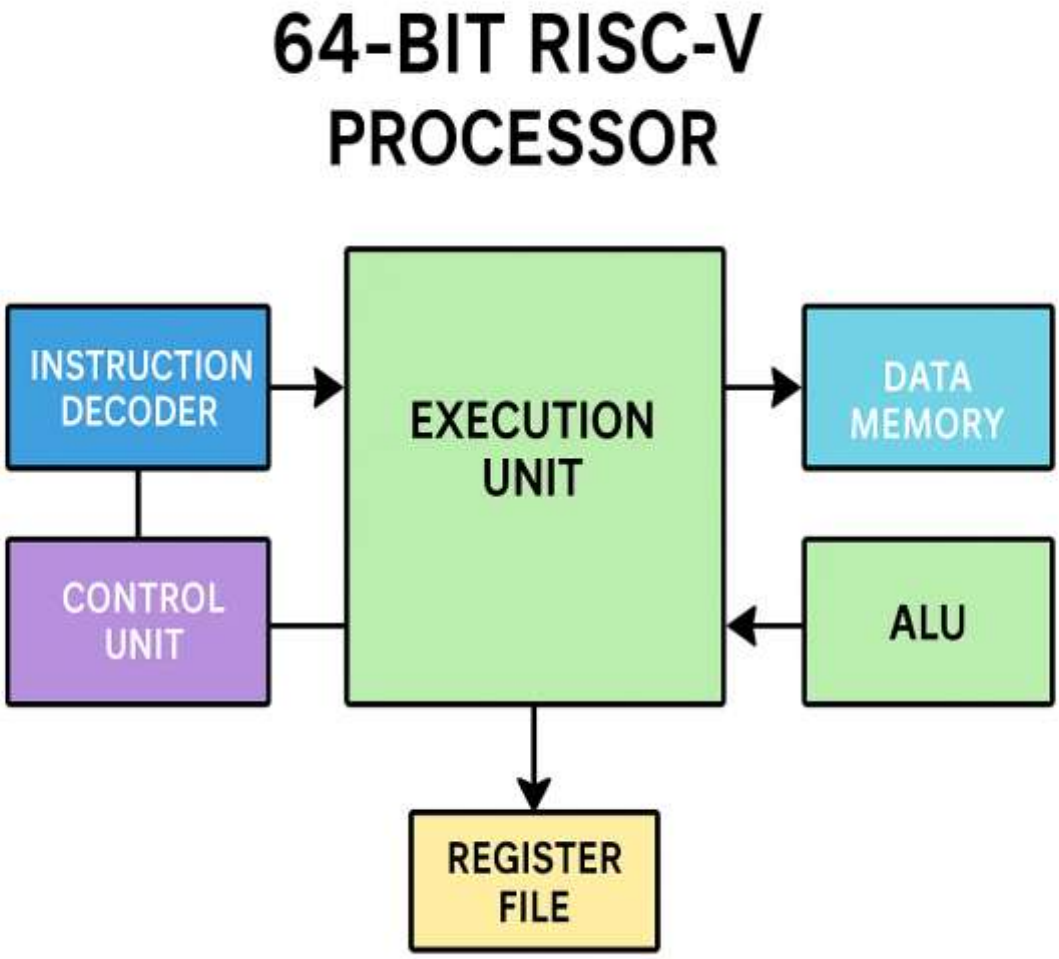

Fig.06 64-bit RISC-V Processor

CHAPTER 4

SOFTWARE REQUIREMENTS

4.1 Overview of Xilinx Software

Xilinx Vivado is a comprehensive and advanced integrated development environment (IDE) and design toolset primarily used for designing and programming Xilinx FPGAs (Field-Programmable Gate Arrays) and SoCs (System on Chips).

4.2 About Software:

FPGA and SoC Development: Vivado is a versatile software tool that is primarily used for the design, analysis, and programming of Xilinx FPGAs and SoCs. It supports a wide range of Xilinx devices, from entry-level FPGAs to high-end, high-performance chips. Vivado provides support for high-level design entry, allowing engineers and developers to use languages like VHDL, Verilog, and high-level programming languages such as C, C++, and OpenCL to create hardware designs. The toolset includes a large library of pre-designed Intellectual Property (IP)cores that can be easily integrated into your FPGA or SoC design. This simplifies the design process and reduces development time. Vivado performs logic synthesis to convert high-level descriptions of your design into the lower-level logic gates that will be programmed into the FPGA or SoC. It optimizes the design for performance, area, and power consumption.

4.3 Implementation, Placing, and Routing:

Vivado provides tools for physical implementation and place-and-route, which determine the exact arrangement of logic elements and interconnections on the FPGA or SoC. This step is crucial for optimizing the design for target hardware. The software includes timing analysis tools to ensure that your design meets the required performance specifications and that signals propagate correctly through the device. Vivado offers debugging and verification tools, including hardware debugging through JTAG, simulation capabilities, and interfaces with third-party simulation tools. We can use Vivado to generate bit streams that configure the FPGA, enabling it to perform the desired functionality. Vivado supports partial and dynamic reconfiguration, allowing you to change the functionality of specific regions of the FPGA while other parts remain operational.

Xilinx Vivado is a comprehensive software solution for FPGA and SoC design, offering a wide range of features and tools to support hardware development and optimization. It is widely used in various industries, including aerospace, telecommunications, automotive, and many more, for creating custom hardware solutions.

4.4 Working with the Vivado IDE:

The Vivado Integrated Design Environment (IDE) can be used in both Project Mode and Non-Project Mode. The Vivado IDE provides an interface to assemble, implement, and validate your design and IP. Opening a design loads the current design netlist, applies design constraints, and fits the design onto the target device.

4.5 Launching of Vivado:

Select Start → All

Programs → Xilinx Design Tools → Vivado → Vivado.

Note: You can also double-click the Vivado IDE shortcut icon on your desktop.

Fig 7: Vivado IDE Desktop Icon

4.6 Creating Projects:

The Vivado Design Suite supports different types of projects for different design purposes. For example, you can create a project with RTL sources or synthesis. netlists from third-party synthesis providers. You can also create empty I/O planning projects to enable device exploration and early pin planning. The Vivado IDE only displays commands relevant to the selected project type.

In the Vivado IDE, the Create Project wizard walks you through the process of creating a project. The wizard enables you to define the project, including the project name, the location in which to store the project, the project type (for example, RTL, netlist, and so forth), and the target part. You can add different types of sources, such as RTL, IP, Block designs, XDC or SDC constraints, simulation test benches, DSP modules from System Generator as IP, or Vivado High-Level Synthesis (HLS), and design documentation. When you select sources, you can determine whether to reference the source in its original location or to copy the source into the project directory. The Vivado Design Suite tracks the time and date stamp of each file and reports status. If files are modified, you are alerted to out-of-date source or design status. For more information, see this link in the Vivado Design Suite User Guide: System-Level Design Entry.

4.7 Different Types of Projects:

The Vivado Design Suite allows for different design entry points depending on your source file types and design tasks. The following are the different types of projects you can use to facilitate those tasks:

RTL Project: You can add RTL source files and constraints, configure IP with the Vivado IP catalog, create IP subsystems with the Vivado IP integrator, synthesize and implement the design, and perform design planning and analysis.

Post-Synthesis Project: You can import third-party netlists, implement the design, and perform design planning and analysis.
I/O Planning Project: You can create an empty project for use with early I/O planning and device exploration before having RTL sources.
Imported Project: You can import existing project sources from the ISE Design Suite, Xilinx Synthesis Technology (XST), or Synopsys Synplify.
Example Project: You can explore several example projects, including Zynq-7000 SoC or MicroBlaze embedded designs with available Xilinx evaluation boards.

4.8 Understanding The Flow Navigator:

The Flow Navigator (shown in the following figure) provides control over the major design process tasks, such as project configuration, synthesis, implementation, and bitstream generation. The commands and options available in the Flow Navigator depend on the status of the design. Unavailable steps are grayed out until required design tasks are completed.

The Flow Navigator (shown in the following figure) differs when working with projects created with third-party netlists. For example, system-level design entry, IP, and synthesis options are not available.

Fig 08: Flow Navigator

Flow Navigator for Third-Party Netlist Project. As the design tasks are complete, you can analyze the RTL by performing Linting on it and specifying the waive-specific violations in the current design. After cleaning up the design, you can open the resulting designs to analyze the results and apply constraints. In the Flow Navigator When you open a design, the Flow Navigator shows a set of commonly used commands for the applicable phase of the design flow. Selecting any of these commands in the Flow Navigator opens the design, if it is not already opened, and performs the operation. For example, the following figure shows the commands related to synthesis.

The Vivado IDE Sources window (shown in the following figure) provides automated source file management. The window has several views to display the sources using different methods. When you open or modify a project, the Sources window updates the status of the project sources. A quick compilation of the design source files is performed and the sources appear in the Compile Order view of the Sources window in the order they will be compiled by the downstream tools. Any potential issues with the compilation of the RTL hierarchy are shown as well as reported in the Message window.

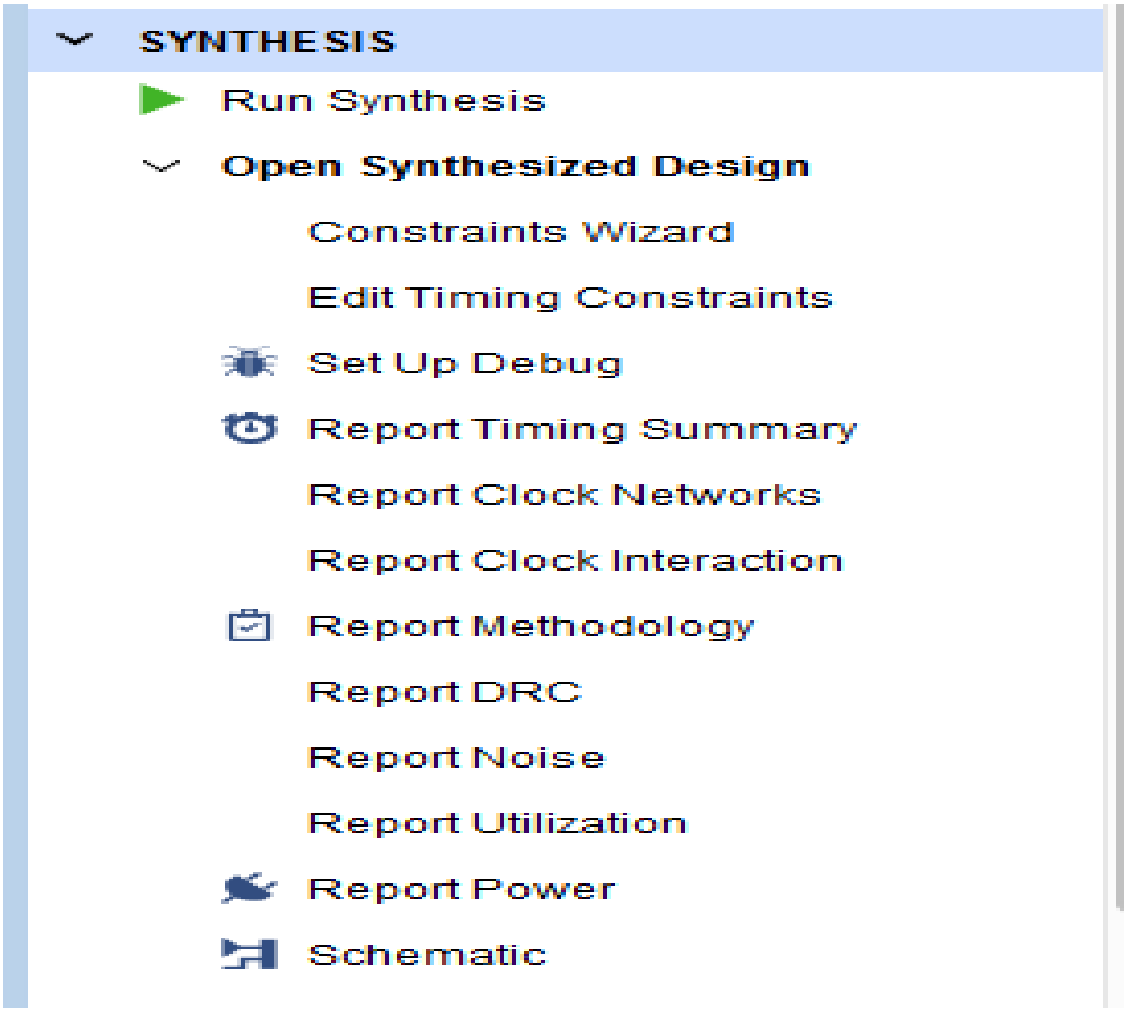

Fig 09: Synthesis Section in the Flow Navigator

4.9 Automated Hierarchical Source File Compilation and Management:

The Vivado IDE Sources window (shown in the following figure) provides automated source file management. The window has several views to display the sources using different methods. When you open or modify a project, the Sources window updates the status of the project sources.

A quick compilation of the design source files is performed and the sources appear in the Compile Order view of the Sources window in the order they will be compiled by the downstream tools. Any potential issues with the compilation of the RTL hierarchy are shown as well as reported in the Message window. For more information on sources, see this link in the Vivado

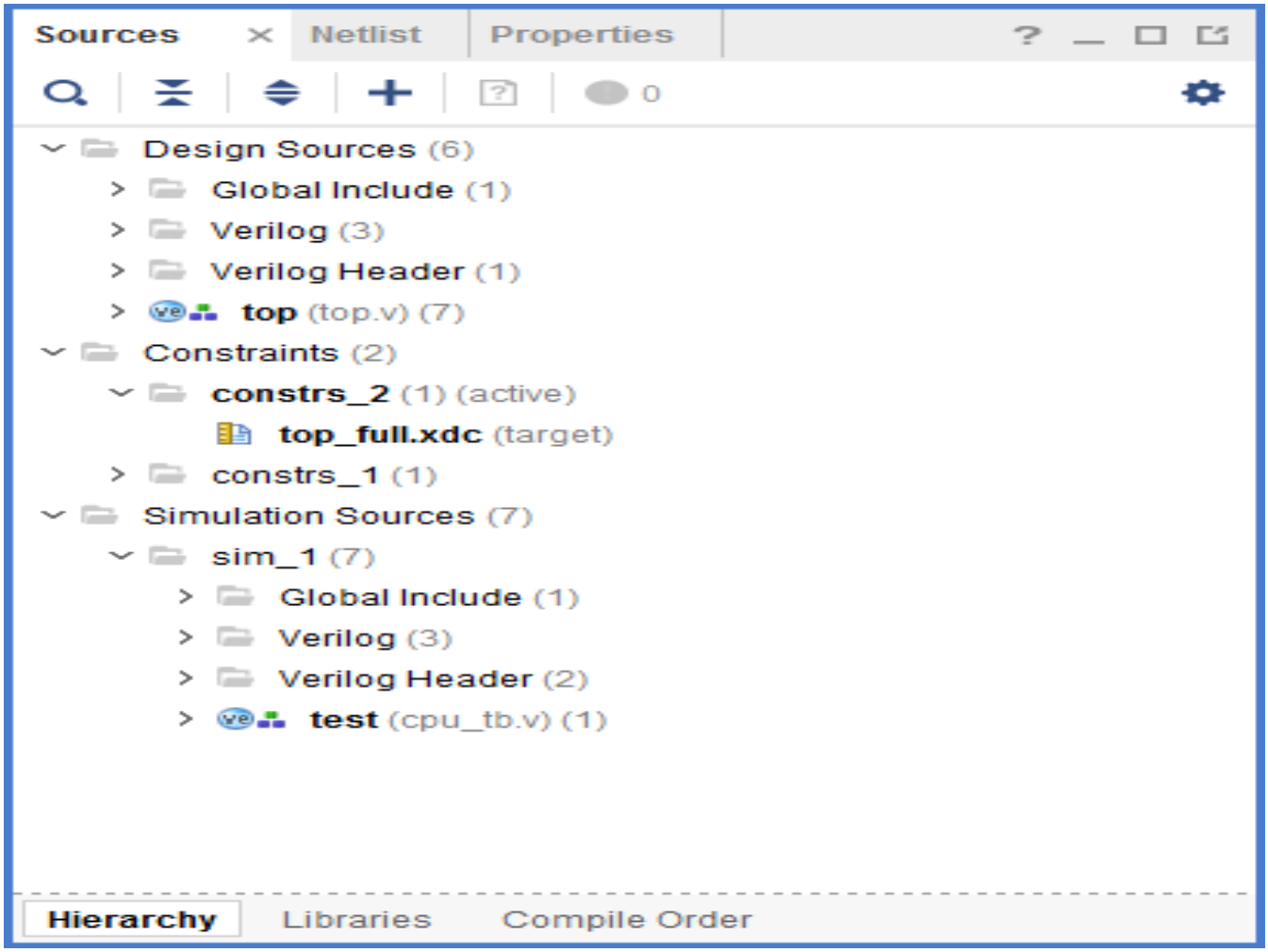

Fig 10: Hierarchical Source

4.10 Simulation Flow:

The Vivado Design Suite supports both integrated simulation, which allows you to run the simulator from within the Vivado IDE, and batch simulation, which allows you to generate a script from the Vivado tools to run simulation on an external verification environment.

Integrated Simulation:

The Vivado IDE provides full integration with the Vivado simulator, and all supported third-party simulators. In this flow, the simulator is called from within the Vivado IDE, and you can compile and simulate the design easily with a push of a button, or with the launch simulation T+cl command.

4.11 Running Logic Synthesis And Implementation

- Logic Synthesis

Vivado synthesis enables you to configure, launch, and monitor synthesis runs. The Vivado IDE displays the synthesis results and creates report files. You can select synthesis warnings and errors from the Messages window to highlight the logic in the RTL source files.

You can launch multiple synthesis runs concurrently or serially. On a Linux system, you can launch runs locally or on remote servers. With multiple synthesis runs, Vivado synthesis creates multiple netlists that are stored with the Vivado Design Suite project. You can open different versions of the synthesized netlist in the Vivado IDE to perform device and design analysis. You can also create constraints for I/O pin planning, timing, floor planning, and implementation. The most comprehensive list of DRCs is available after a synthesized netlist is produced, when clock and clock logic are available for analysis and placement.

- Implementation

Vivado implementation enables you to configure, launch, and monitor implementation runs. You can experiment with different implementation options and create your own reusable strategies for implementation runs. For example, you can create strategies for quick run times, improved system performance, or area optimization. As the runs complete, the implementation run results display, and report files are available.

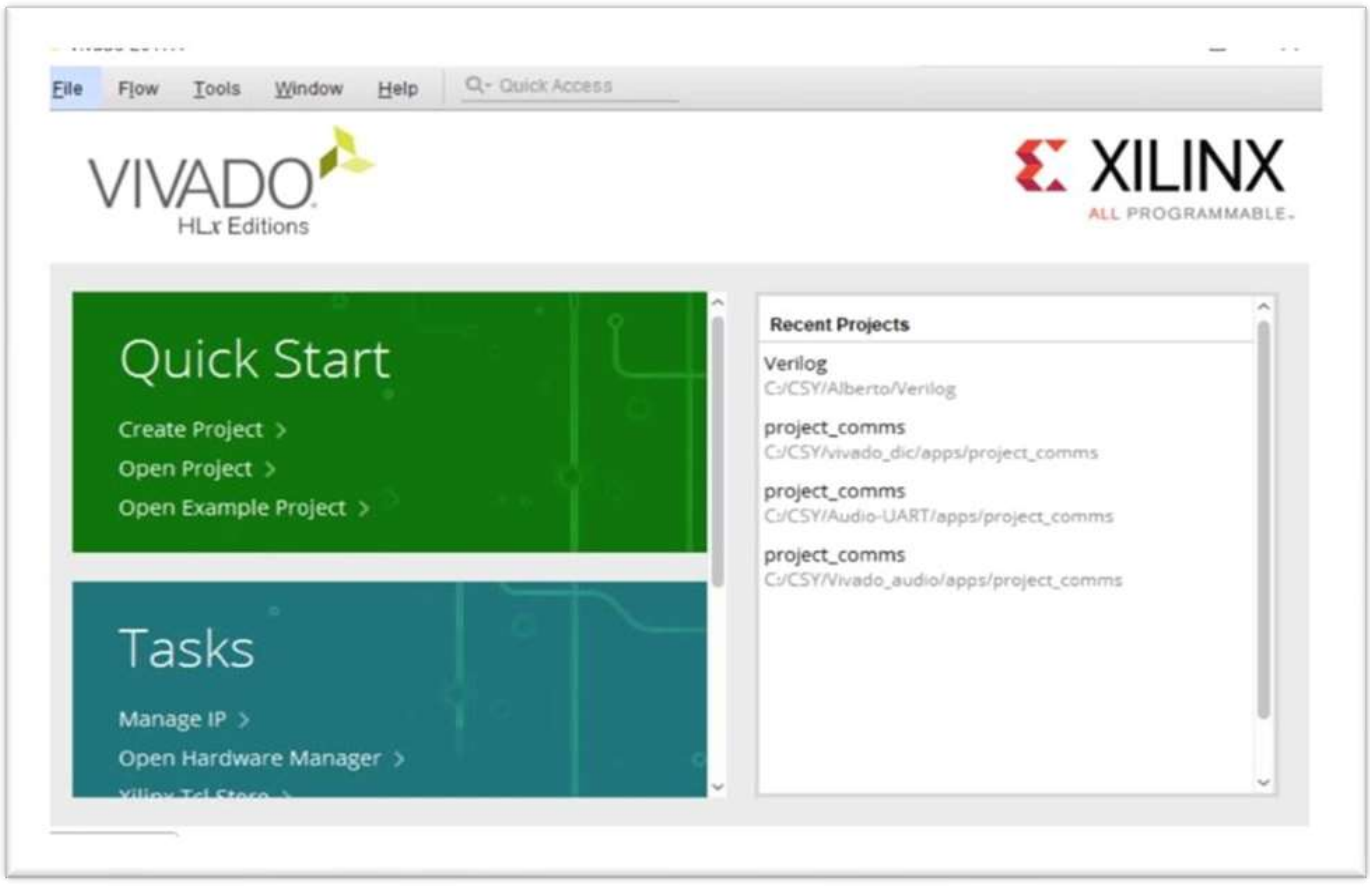

Fig.11: Interface of Xilinx Vivado

4.12 Applications of Vivado Software:

Vivado is primarily used for the development of FPGA and SoC designs. It provides a robust platform for hardware design, simulation, synthesis, and implementation, making it a go-to tool for creating custom digital logic circuits and embedded systems. Vivado is extensively used in DSP applications, where high-performance signal processing is required. It can accelerate the design and implementation of algorithms on FPGAs for applications like image and speech processing.

- Aerospace and Defense:

In industries such as aerospace and defense, Vivado is used to develop high-reliability systems. FPGAs offer flexibility and can be rapidly reconfigured, making them suitable for applications like radar, communications, and secure data processing. Vivado also plays a vital role in communication systems, where FPGAs are used to implement protocol and interface converters, high-speed data processing, and encryption/decryption modules. In the automotive industry, FPGAs and SoCs designed with Vivado are used for applications like advanced driver assistance systems (ADAS), infotainment, and in-vehicle networking.

- Industrial Automation:

Vivado is employed in industrial automation for developing control systems, data acquisition, and processing solutions. It offers real-time performance and customization options for various industrial applications. Vivado is widely used in academia for research, teaching, and hands-on learning in digital design, FPGA programming, and embedded systems. It helps students and researchers gain practical experience in FPGA technology. As IoT and edge computing applications grow, Vivado is used to design FPGA and SoC solutions that can process data at the network edge efficiently, enabling real-time analytics and decision-making.

- System-Level Design and IP Integration:

Vivado offers the IP Integrator, which allows users to create complex systems by connecting pre-built IP cores and custom modules graphically.

- Embedded System Design:

Supports embedded design workflows, such as creating hardware platforms for embedded processors. Can be used along with Vitis or SDK to develop and debug C/C++ software for embedded systems.

- Hardware Debugging:

Provides debugging tools like Vivado Logic Analyzer and ILA (Integrated Logic Analyzer) cores for real-time testing on an FPGA. Useful for observing signals and debugging logic errors on actual hardware.

- FPGA and SoC Design

Vivado is primarily used to design, simulate, and implement digital circuits on Xilinx FPGAs and System-on-Chip (SoC) devices.Supports devices such as Zynq-7000, Virtex, Kintex, Artix, and Spartan series.

- Research and Academic Projects

Extensively used in universities and research labs for projects involving digital design, computer architecture, signal processing, machine learning accelerators, etc.

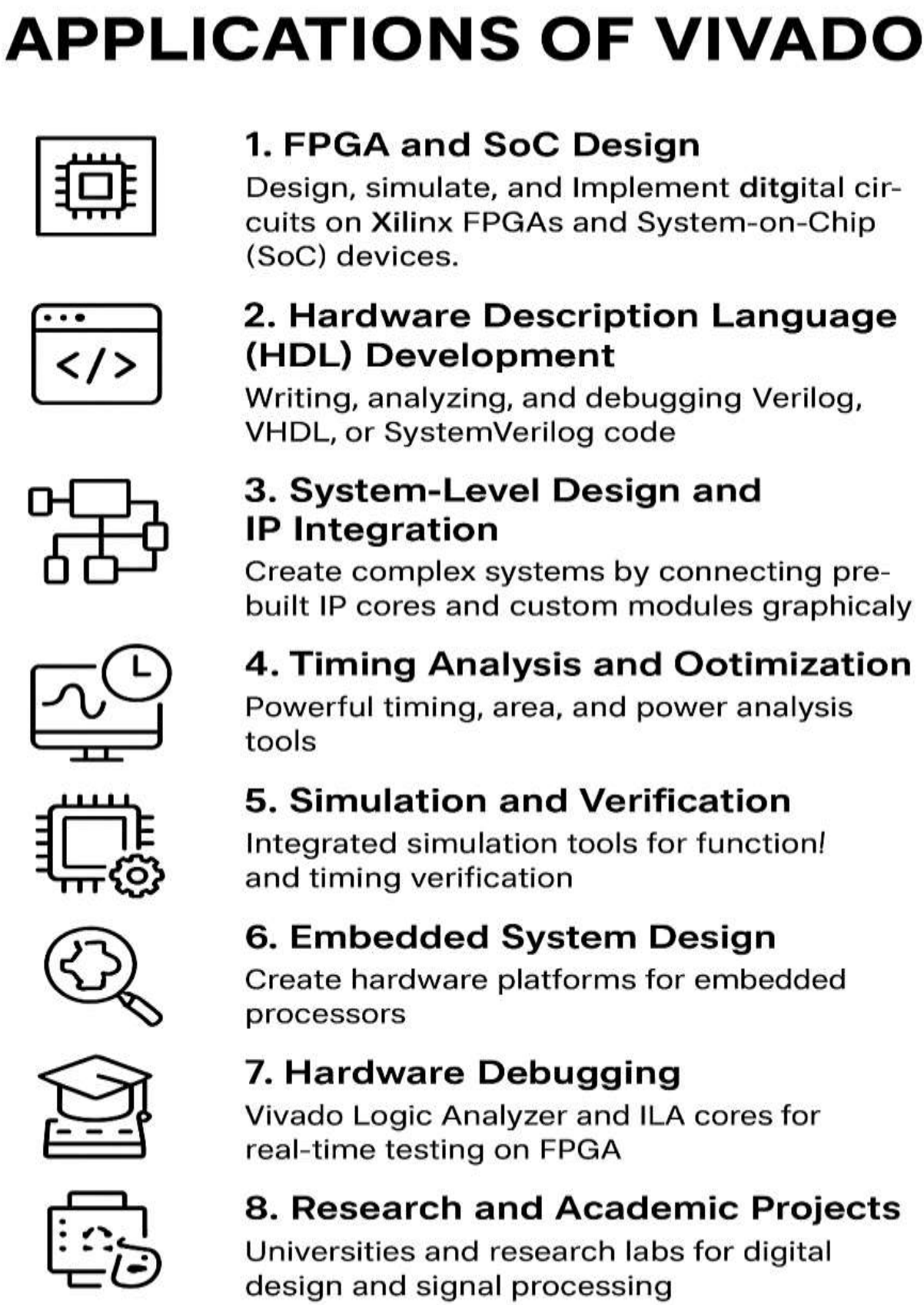

Fig.12: Applications of Vivado

CHAPTER 5

SOURCE CODE

5.1 Verilog Codes:

5.1.1 64-Bit ALU:

```
module alunit64bit(
   input wire  [63:0] op_a,
   input wire  [63:0] op_b,
   input wire  [3:0]  alu_op,
   output reg  [63:0] result,
   output wire        zero,
   output wire        less_than
   );
localparam [3:0]
         ALU_ADD  = 4'd0,  // Add
         ALU_SUB  = 4'd1,  // Subtract
         ALU_AND  = 4'd2,  // Bitwise AND
         ALU_OR   = 4'd3,  // Bitwise OR
         ALU_XOR  = 4'd4,  // Bitwise XOR
         ALU_SLL  = 4'd5,  // Shift left logical
         ALU_SRL  = 4'd6,  // Shift right logical
         ALU_SRA  = 4'd7,  // Shift right arithmetic
         ALU_SLT  = 4'd8,  // Set on less than (signed)
         ALU_SLTU = 4'd9,  // Set on less than (unsigned)
         ALU_PASS_B = 4'd10; // Pass op_b (for immediate values, e.g., LUI, AUIPC)

      // Combinational logic for the ALU operations
      always @(*) begin
         case (alu_op)
            ALU_ADD:    result = op_a + op_b;
            ALU_SUB:    result = op_a - op_b;
            ALU_AND:    result = op_a & op_b;
            ALU_OR:     result = op_a | op_b;
            ALU_XOR:    result = op_a ^ op_b;
            ALU_SLL:    result = op_a << op_b;
            ALU_SRL:    result = op_a >> op_b;
            ALU_SRA:    result = $signed(op_a) >>> op_b;
            ALU_SLT:    result = ($signed(op_a) < $signed(op_b)) ? 64'd1 : 64'd0;
            ALU_SLTU:   result = (op_a < op_b) ? 64'd1 : 64'd0;
            ALU_PASS_B: result = op_b;
            default:    result = 64'hX; // Undefined result for unhandled operations
        endcase
      end
 // Output Flags.
      assign zero = (result == 64'b0);
      // The less_than flag is useful for signed branches.
      assign less_than = ($signed(op_a) < $signed(op_b));

endmodule
```

5.1.2 64-Bit Register File Unit:

```
module registerfile_64bit(
   input  wire       clk,
   input  wire       rst,
   input  wire       reg_write_en,
   input  wire [4:0]  rs1,
   input  wire [4:0]  rs2,
   input  wire [4:0]  rd,
   input  wire [63:0] reg_write_data,
   output wire [63:0] rs1_read_data,
   output wire [63:0] rs2_read_data
   );

      // Declare the 64-bit wide, 32-entry register bank
      reg [63:0] registers [0:31];

      // Declare the loop variable for reset
      integer i;

      // Read Logic: Asynchronous reads
      // Reads are combinational. If the address is 0, the output is hardwired to 0.

      assign rs1_read_data = (rs1 == 5'b0) ? 64'b0 : registers[rs1];
      assign rs2_read_data = (rs2 == 5'b0) ? 64'b0 : registers[rs2];

      // Write Logic: Synchronous writes
      // The write operation occurs on the positive edge of the clock.

      always @(posedge clk or posedge rst) begin
         if (rst) begin

            // Asynchronous reset: initialize all registers to zero.

            for (i = 0; i < 32; i = i + 1) begin
               registers[i] <= 64'b0;
            end
         end else begin
            if (reg_write_en) begin

               // Only write if the destination register is not x0 (register 0).

               if (rd != 5'b0) begin
                  registers[rd] <= reg_write_data;
               end
            end
         end
      end
endmodule
```

5.1.3 64-Bit Memory Unit:

```
module memu_64bit(
   input  wire         clk,
   input  wire         rst,
   input  wire [63:0]  ex_mem_adder,
   input  wire [63:0]  ex_mem_wdata,
   input  wire [63:0]  ex_mem_op_type,
   output reg          mem_req,
   output reg          mem_write_en,
   output reg  [63:0]  mem_adder,
   output reg  [63:0]  mem_wdata,
   input  wire [63:0]  mem_rdata,
   input  wire         mem_ready,
   output reg  [63:0]  mem_wb_rdata,
   output reg          mem_wb_valid
   );
   // Memory Operation Types (for ex_mem_op_type)
      // RV64I base ISA
      localparam OP_NOP  = 3'b000;
      localparam OP_LD   = 3'b001; // Load Doubleword
      localparam OP_LW   = 3'b010; // Load Word (sign-extended)
      localparam OP_LWU  = 3'b011; // Load Word (unsigned)
      localparam OP_LH   = 3'b100; // Load Halfword (sign-extended)
      localparam OP_LHU  = 3'b101; // Load Halfword (unsigned)
      localparam OP_LB   = 3'b110; // Load Byte (sign-extended)
      localparam OP_LBU  = 3'b111; // Load Byte (unsigned)

      localparam OP_SD   = 3'b001; // Store Doubleword (reusing opcode for simplicity)
      localparam OP_SW   = 3'b010; // Store Word
      localparam OP_SH   = 3'b100; // Store Halfword
      localparam OP_SB   = 3'b110; // Store Byte

      // Internal register for read data
      reg [63:0] rdata_reg;

      always @(posedge clk or posedge rst) begin
         if (rst) begin
            mem_req       <= 1'b0;
            mem_write_en  <= 1'b0;
            mem_adder     <= 64'b0;
            mem_wdata     <= 64'b0;
            mem_wb_rdata  <= 64'b0;
            mem_wb_valid  <= 1'b0;
            rdata_reg     <= 64'b0;
         end else begin
            mem_req <= 1'b0;
            mem_wb_valid <= 1'b0;
            rdata_reg     <= 64'b0;
```

```
end else begin
        mem_req <= 1'b0;
        mem_wb_valid <= 1'b0;

        // Handle memory access based on operation type
        case (ex_mem_op_type)
          // Load Operations
          OP_LD, OP_LW, OP_LWU, OP_LH, OP_LHU, OP_LB, OP_LBU: begin
            mem_req       <= 1'b1;
            mem_write_en  <= 1'b0;
            mem_adder     <= ex_mem_adder;

            if (mem_ready) begin
              rdata_reg <= mem_rdata;
              case (ex_mem_op_type)
                OP_LD: begin
                  mem_wb_rdata <= rdata_reg;
                  mem_wb_valid <= 1'b1;
                end
                OP_LW: begin // Sign-extended Word
                  mem_wb_rdata <= {{32{rdata_reg[31]}}, rdata_reg[31:0]};
                  mem_wb_valid <= 1'b1;
                end
                OP_LWU: begin // Unsigned Word
                  mem_wb_rdata <= {{32'b0}, rdata_reg[31:0]};
                  mem_wb_valid <= 1'b1;
                end
                OP_LH: begin // Sign-extended Halfword
                  mem_wb_rdata <= {{48{rdata_reg[15]}}, rdata_reg[15:0]};
                  mem_wb_valid <= 1'b1;
                end
                OP_LHU: begin // Unsigned Halfword
                  mem_wb_rdata <= {{48'b0}, rdata_reg[15:0]};
                  mem_wb_valid <= 1'b1;
                end
                OP_LB: begin // Sign-extended Byte
                  mem_wb_rdata <= {{56{rdata_reg[7]}}, rdata_reg[7:0]};
                  mem_wb_valid <= 1'b1;
                end
                OP_LBU: begin // Unsigned Byte
                  mem_wb_rdata <= {{56'b0}, rdata_reg[7:0]};
                  mem_wb_valid <= 1'b1;
                end
              endcase
            end
          end
```

```
// Store Operations (assuming simple opcode mapping for this example)
            OP_SD: begin
               mem_req        <= 1'b1;
               mem_write_en    <= 1'b1;
               mem_adder       <= ex_mem_adder;
               mem_wdata       <= ex_mem_wdata;
            end
            OP_SW: begin
               mem_req        <= 1'b1;
               mem_write_en    <= 1'b1;
               mem_adder       <= ex_mem_adder;
               mem_wdata       <= {{32'b0}, ex_mem_wdata[31:0]}; // Mask for 32-bit write
            end
            OP_SH: begin
               mem_req        <= 1'b1;
               mem_write_en    <= 1'b1;
               mem_adder       <= ex_mem_adder;
               mem_wdata       <= {{48'b0}, ex_mem_wdata[15:0]}; // Mask for 16-bit write
            end
            OP_SB: begin
               mem_req        <= 1'b1;
               mem_write_en    <= 1'b1;
               mem_adder       <= ex_mem_adder;
               mem_wdata       <= {{56'b0}, ex_mem_wdata[7:0]}; // Mask for 8-bit write
            end

            default: begin
               // NOP
               mem_req        <= 1'b0;
               mem_write_en    <= 1'b0;
            end
         endcase
      end
   end
endmodule
```

CHAPTER 6

SIMULATION RESULTS

6.1 Simulation Steps of 64-Bit ALU, REGISTER FILE UNIT AND MEMORY UNIT

STEP 1:Create a New Project

- Open Xilinx Vivado.
- Click on New Project.
- Set project name and location.
- Choose RTL Project
- Select the target part (e.g., Artix-7 xc7a100tcsg324-3).
- Click Finish.

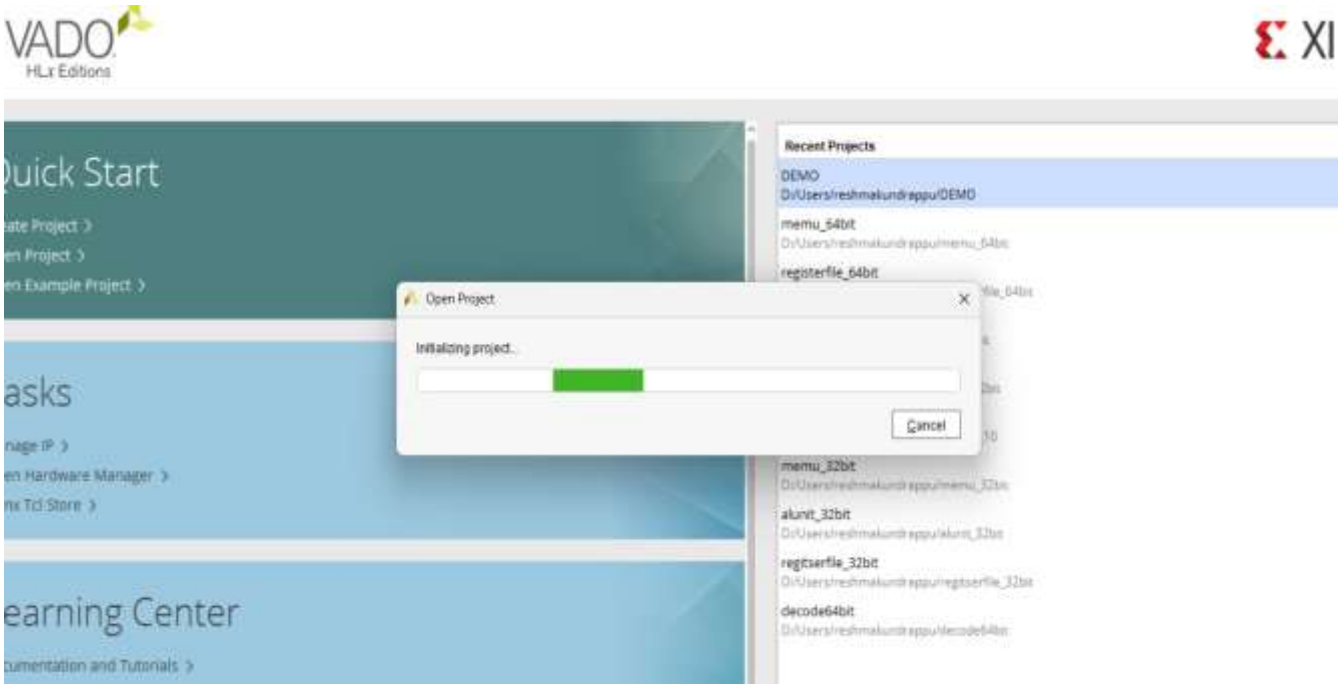

Fig.13: Creating a new project

STEP 2: Add Design Sources

- Go to add sources in the flow navigator.
- Select and add, create design sources.
- Create a New Verilog file.

STEP 3: Define Inputs and Output Ports

- Port declaration will be done by assigning inputs and outputs
- This step is followed by the addition of Bus, i.e, MSB and LSB bits if required.
- Click on finish
- After it gets updated, click on the three dots below design sources to access the editor window

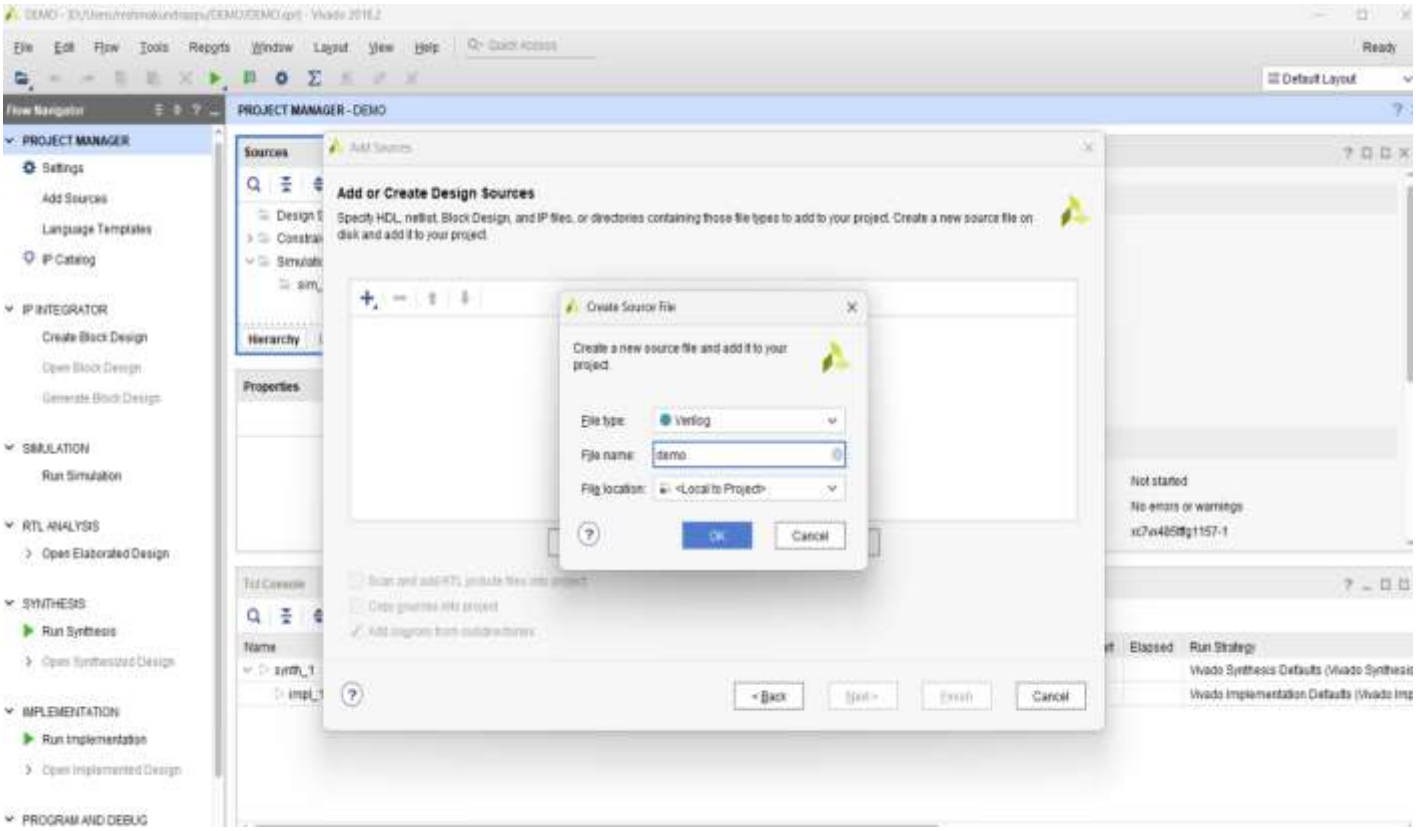

Fig.14: Adding Design Sources.

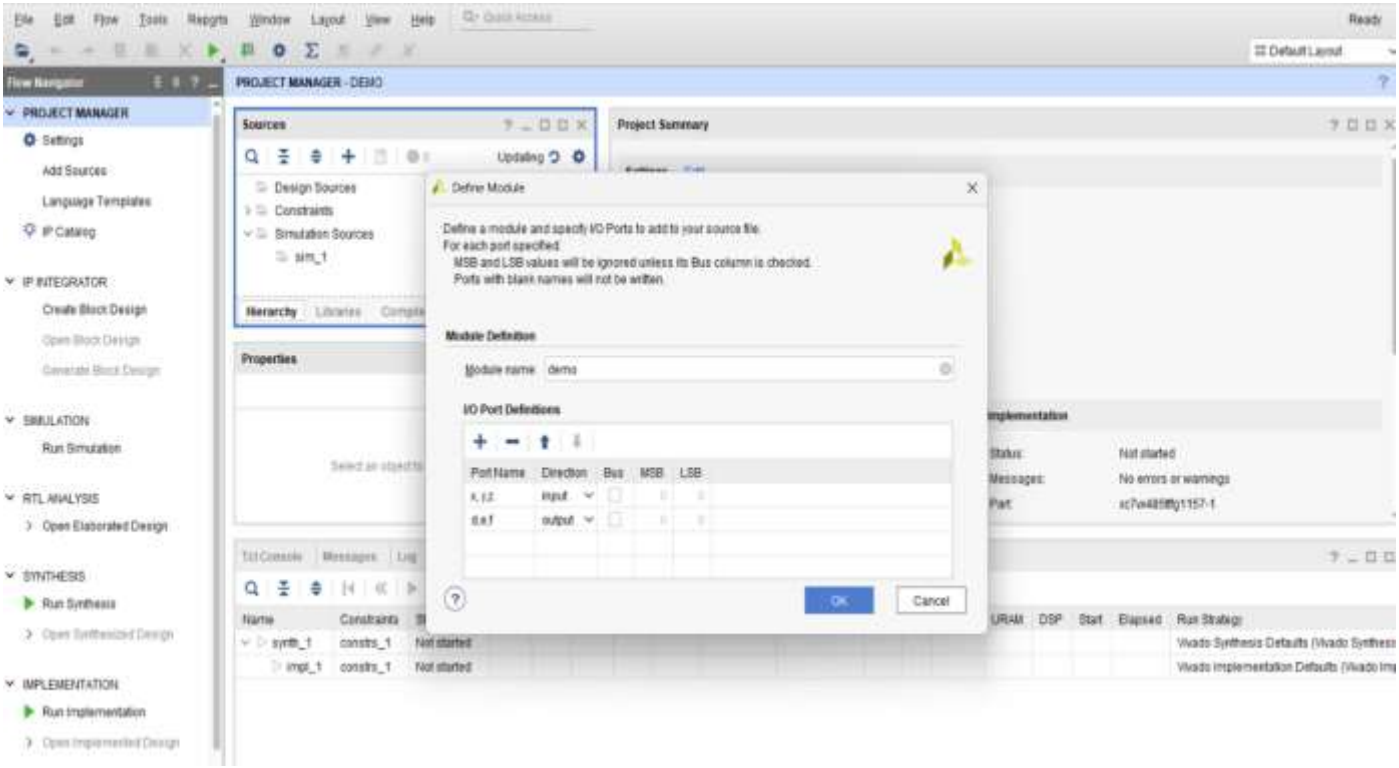

Fig.15: Declaration of Ports

STEP 4: Force Constant and Force Clock:

- Write the code for each block.(as a separate project only).
- Check for errors and correct them.
- Click on run behavioral simulation.
- Now assign Force Clock and Force Constant values.

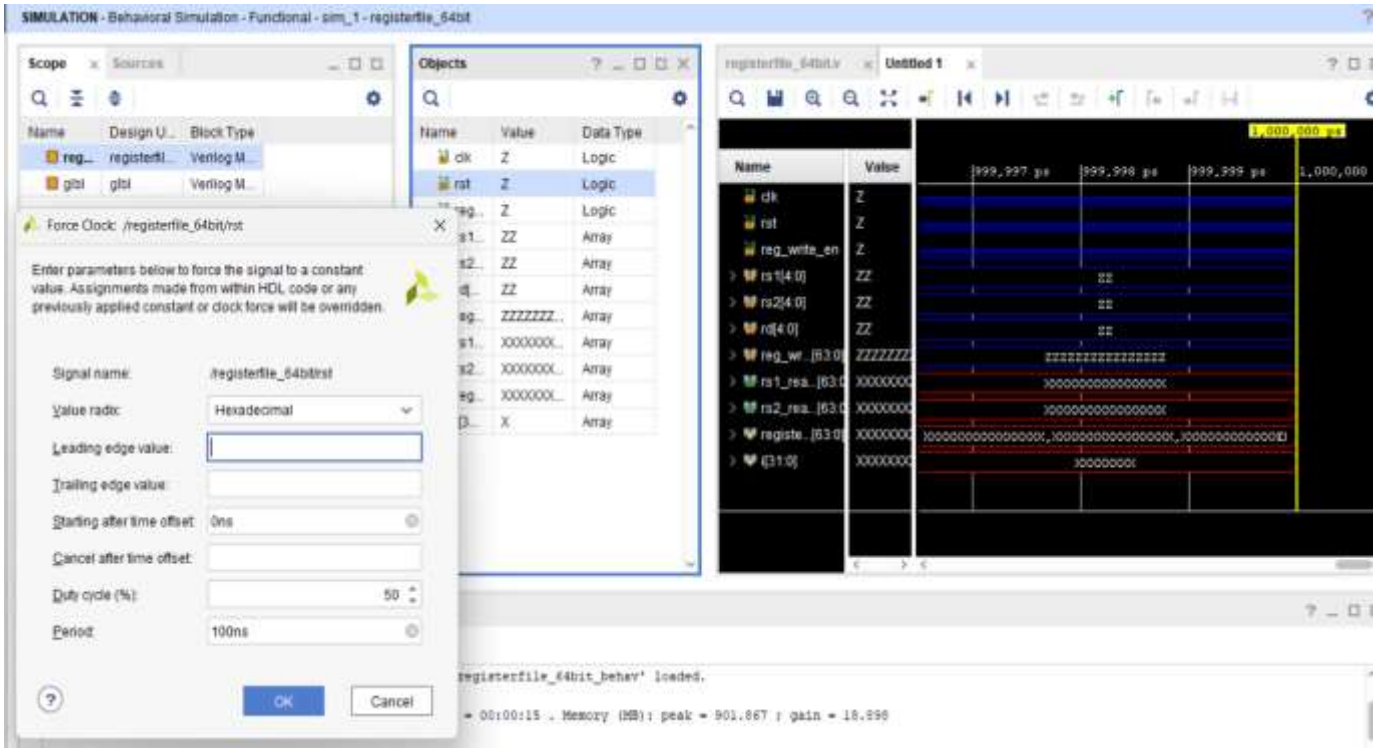

Fig.16: Force Clock

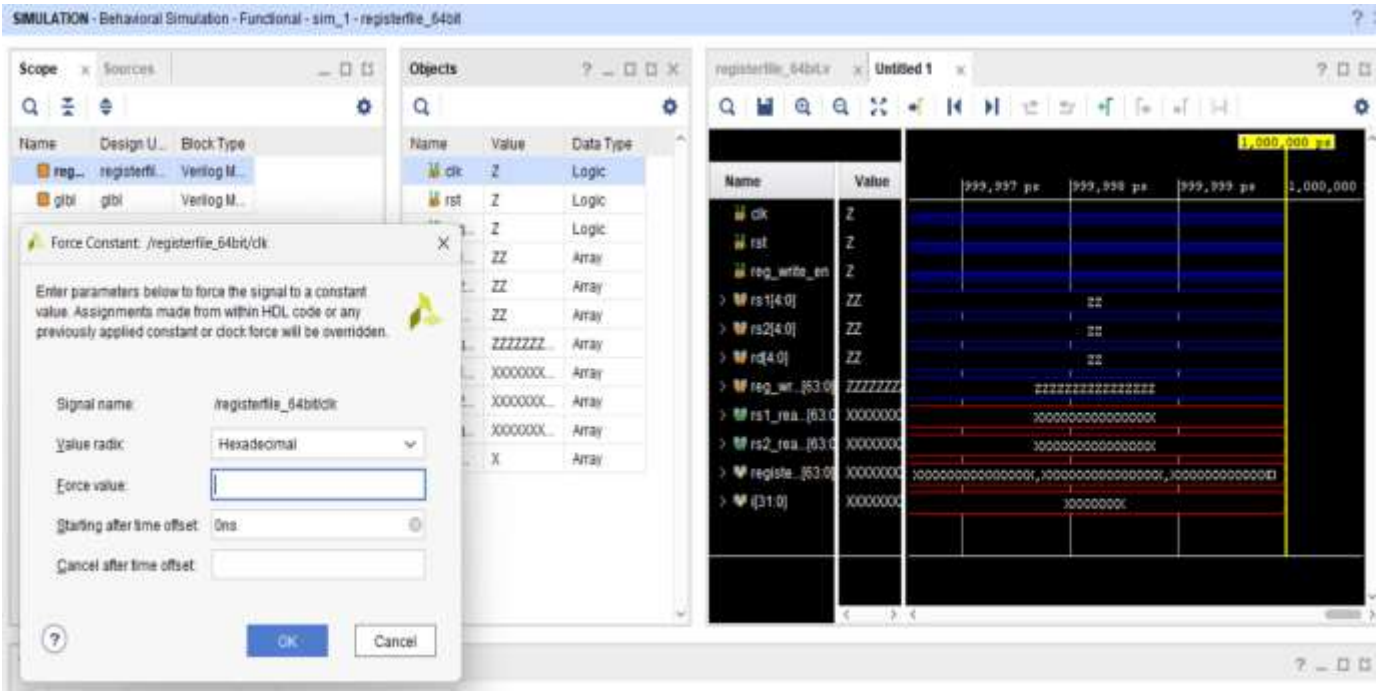

Fig.17: Force Constant

STEP 5: Force Constant and Force Clock

- Write the code for each block.(as a separate project only).
- Check for errors and correct them.
- Click on run behavioral simulation.
- Now assign Force Clock and Force Constant values.

STEP 6: Clock Signal Characteristics

Leading Edge: The clock transitions from low to high, where most flip-flops capture data.
Trailing Edge: The clock transitions from high to low, where some circuits may respond or update their states.

STEP 7: Analyze Waveforms

- Check the waveform viewer for inputs and outputs.
- Ensure correct sum and carry-out values based on the inputs.

STEP 8: Verify Simulation Results

- Confirm that the outputs match expected results.

CHAPTER 7

VERIFICATION OF RESULTS - WAVEFORMS

7.1 Arithmetic and Logical Unit:

This module alunit64bit implements a 64-bit Arithmetic Logic Unit (ALU) used in a RISC-V processor datapath.It takes:

- op_a and op_b → two 64-bit operands
- alu_op → a 4-bit control signal selecting which operation to perform

Produces:

- result → 64-bit output
- zero → high when result = 0
- less_than → high when op_a < op_b (signed comparison)

The ALU supports the following operations:

Operation	alu_op	Meaning
ADD	0000 (0)	op_a + op_b
SUB	0001 (1)	op_a - op_b
AND	0010 (2)	op_a & op_b
OR	0011 (3)	op_a | op_b
XOR	0100 (4)	op_a ^ op_b
SLL	0101 (5)	op_a << op_b
SRL	0110 (6)	op_a >> op_b
SRA	0111 (7)	signed shift right
SLT	1000 (8)	(op_a < op_b) ? 1 : 0
SLTU	1001 (9)	unsigned compare
PASS_B	1010 (10)	Pass op_b directly

Table-03 ALU

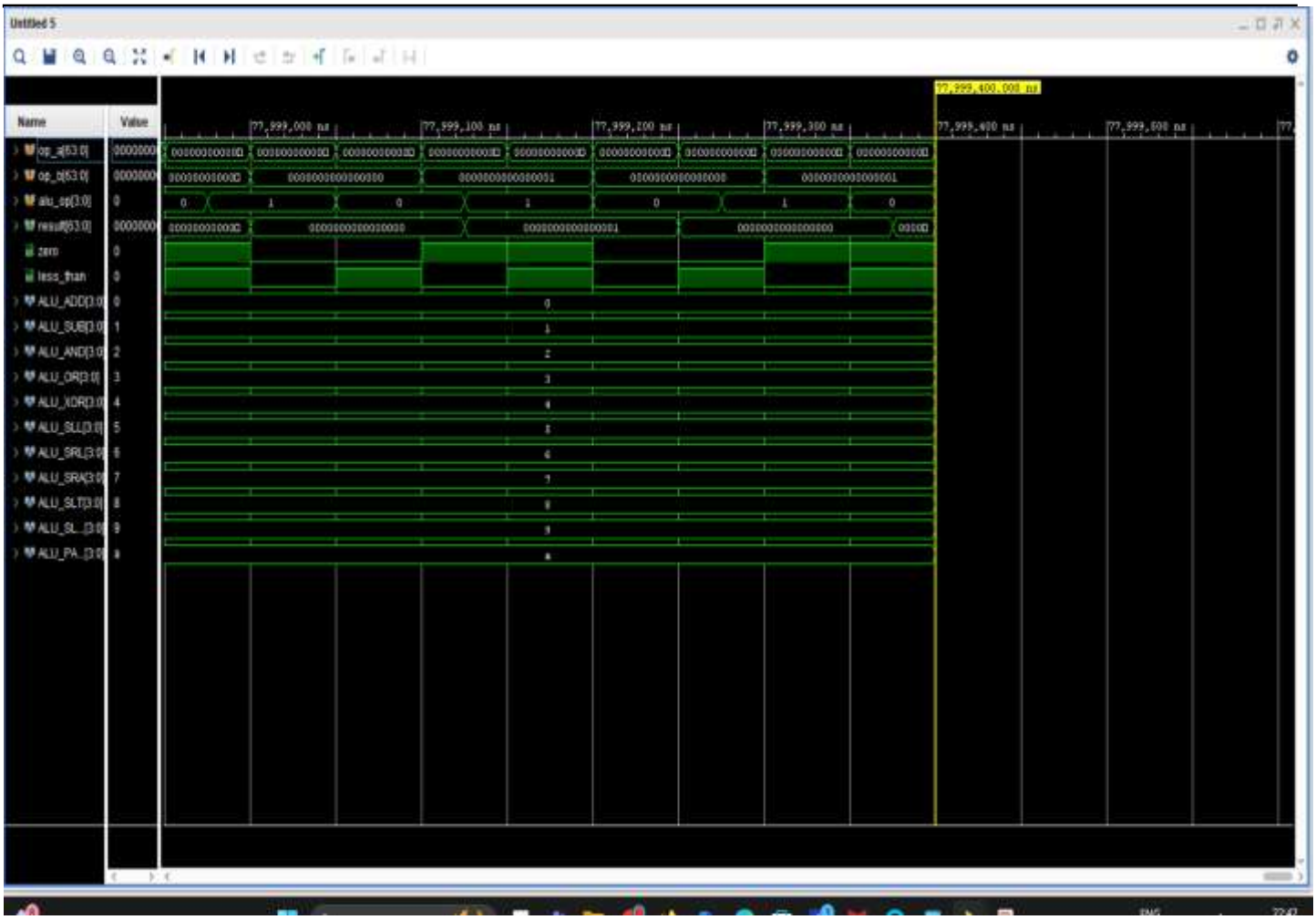

Fig.18: ALU Output Waveform

You can see the ALU codes (2–10) listed in the waveform left panel:
2 (AND) → result = op_a & op_b
3 (OR) → result = op_a | op_b
4 (XOR) → result = op_a ^ op_b
5 (SLL) → logical left shift
6 (SRL) → logical right shift
7 (SRA) → arithmetic right shift
8 (SLT) → result = 1 if op_a < op_b (signed)
9 (SLTU) → result = 1 if op_a < op_b (unsigned)
A (PASS_B) → result = op_b

Example Verification:

- Let's verify one case (ALU_SLTU):
- If op_a = 0xFFFFFFFFFFFFFFFF (unsigned = very large number)
- op_b = 0x0000000000000001,
- then the unsigned compare gives op_a > op_b,
- so **result = 0**.

But for **signed** compare (ALU_SLT):
- op_a = -1 (signed) and op_b = 1,
- so -1 < 1 → **result = 1**.

That's why both signed and unsigned comparisons are separate operations.

ALU Code	Operation	Example (op_a=1, op_b=1)	Result
0	ADD	1+1	2
1	SUB	1–1	0
2	AND	1&1	1
3	OR	1|1	1
4	XOR	1^1	0
5	SLL	1<<1	2
6	SRL	2>>1	1
7	SRA	signed(−2)>>1	−1
8	SLT	1<1 ?	0
9	SLTU	same as SLT for +ve	0
10	PASS_B	op_b	1

Table-4 ALU Operation

7.2 Register File Unit:

The module registerfile_64bit represents the **register file** used in a **RISC-V processor** — it's a memory unit that stores register values (like x0–x31).

Signal	Width	Description
clk	1 bit	Clock signal (writes happen on positive edge)
rst	1 bit	Reset signal (active high; clears registers)
reg_write_en	1 bit	Enables write operation when high
rs1, rs2	5 bits each	Read register addresses
rd	5 bits	Write register address
reg_write_data	64 bits	Data to write into registers[rd]
rs1_read_data, rs2_read_data	64 bits each	Data read from registers rs1 and rs2

Table-5 Register File Unit Description

Signal	Observation	Meaning
clk	Periodic square wave	Regular clock pulses drive synchronous writes.
rst	Initially high (1), then low (0)	Registers are cleared when rst=1.
reg_write_en	Alternates $0 \rightarrow 1 \rightarrow 0$	Controls when data can be written.
rs1, rs2, rd	Show different small binary values (00, 01, etc.)	Selecting different registers for read and write.
rs1_read_data, rs2_read_data	64-bit outputs changing accordingly	Reflect the contents of registers being read.
registers[0:31]	Shows memory contents	Internally stored register values.

Table-6 Register File Unit Operation

- At the beginning (Reset phase):

rst = 1. All registers (registers[0] … registers[31]) become 0000...0000.
rs1_read_data and rs2_read_data show 0 because all registers are cleared.

- After Reset is released:

rst = 0. Now, the register file can accept writes on posedge clk if reg_write_en = 1. At the next rising edge of clk, register [1] gets updated to 1 and whenever rs1 = 00001, the output rs1_read_data = 0000000000000001.

- When Reading Registers:

Because register 1 has data 1 and register 0 is always hardwired to 0 (as per RISC-V rule).

- When Writing to Another Register:

Reading rs1 = 00010 will immediately show this value in the next moment on the waveform.

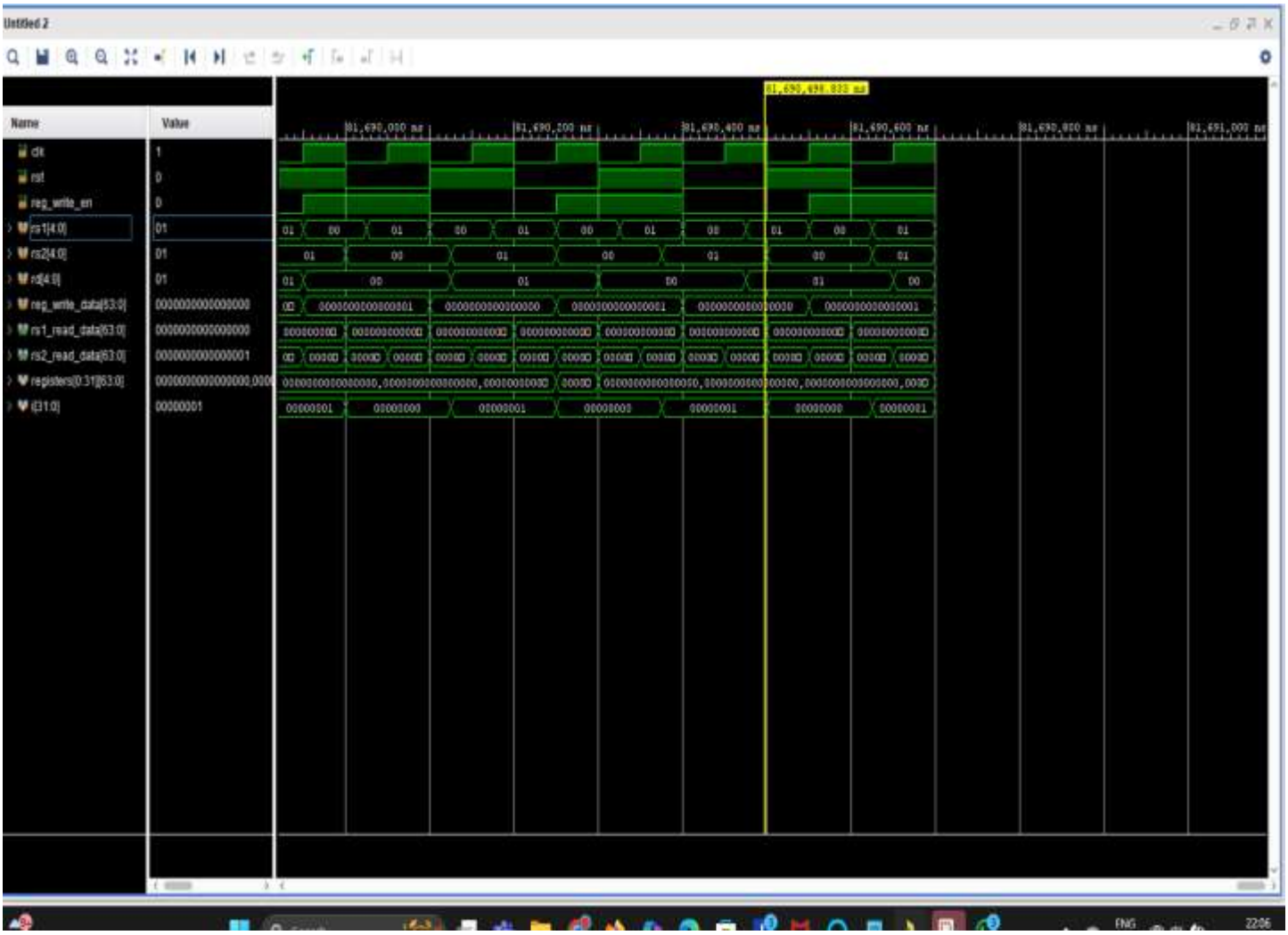

Fig.19: Register File Unit Output Waveform

Example Scenario from the Waveform:

Time	rst	reg_write_en	rs1	rs2	rd	reg_write_data	Action	Result
t_0	1	0	–	–	–	–	Reset active	All regs = 0
t_1	0	1	0	0	1	1	Write 1 → x1	x1 = 1
t_2	0	0	1	0	–	–	Read x1, x0	rs1=1, rs2=0
t_3	0	1	1	0	2	10	Write 10 → x2	x2 = 10
t_4	0	0	2	1	–	–	Read x2, x1	rs1=10, rs2=1

Table-7 Example for Output Waveform

- Reset phase: All registers cleared.
- Write phase: Writes happen only on posedge clk and only when reg_write_en=1.
- Read phase: Asynchronous; immediately shows register content.
- Register x0: Always hardwired to 0 (as per RISC-V spec).

7.3 Memory Unit:

The mem_64bit module is the Memory Access Unit of a RISC-V processor pipeline (the MEM stage).It takes signals from the EX/MEM pipeline register and performs:

- Load operations (reading from memory).
- Store operations (writing to memory).

Signal	Meaning
clk	Clock signal
rst	Reset
ex_mem_adder	Address calculated in Execute stage
ex_mem_wdata	Data to be written (for store)
ex_mem_op_type	Operation code (LD, LW, LH, etc.)
mem_req	Memory request (goes high when a memory op happens)
mem_write_en	Write enable (1 = store, 0 = load)
mem_adder	Address sent to memory
mem_wdata	Data sent to memory for write operations
mem_rdata	Data received from memory during loads
mem_ready	Memory ready signal (acknowledge)
mem_wb_rdata	Data that will go to Write-Back stage
mem_wb_valid	Indicates valid data for write-back
rdata_reg	Internal buffer for memory read data
OP_* lines	The opcodes used (LD, LW, LBU, etc.) — constants for testing

Table-8 Memory Unit

Phases:

- Initial Phase — Reset.
- Load Operations
- Store Operations
- Valid Data Output

For load instructions, you can see mem_wb_valid = 1 when the data is ready. For store instructions, this stays 0, since stores don't produce WB data.

- Memory Ready and Synchronization

Notice how the mem_ready signal is high at specific times:
The memory unit performs read or write only when mem_ready = 1. Until then, data (mem_wb_rdata) does not change. This models a handshake between the CPU and memory.

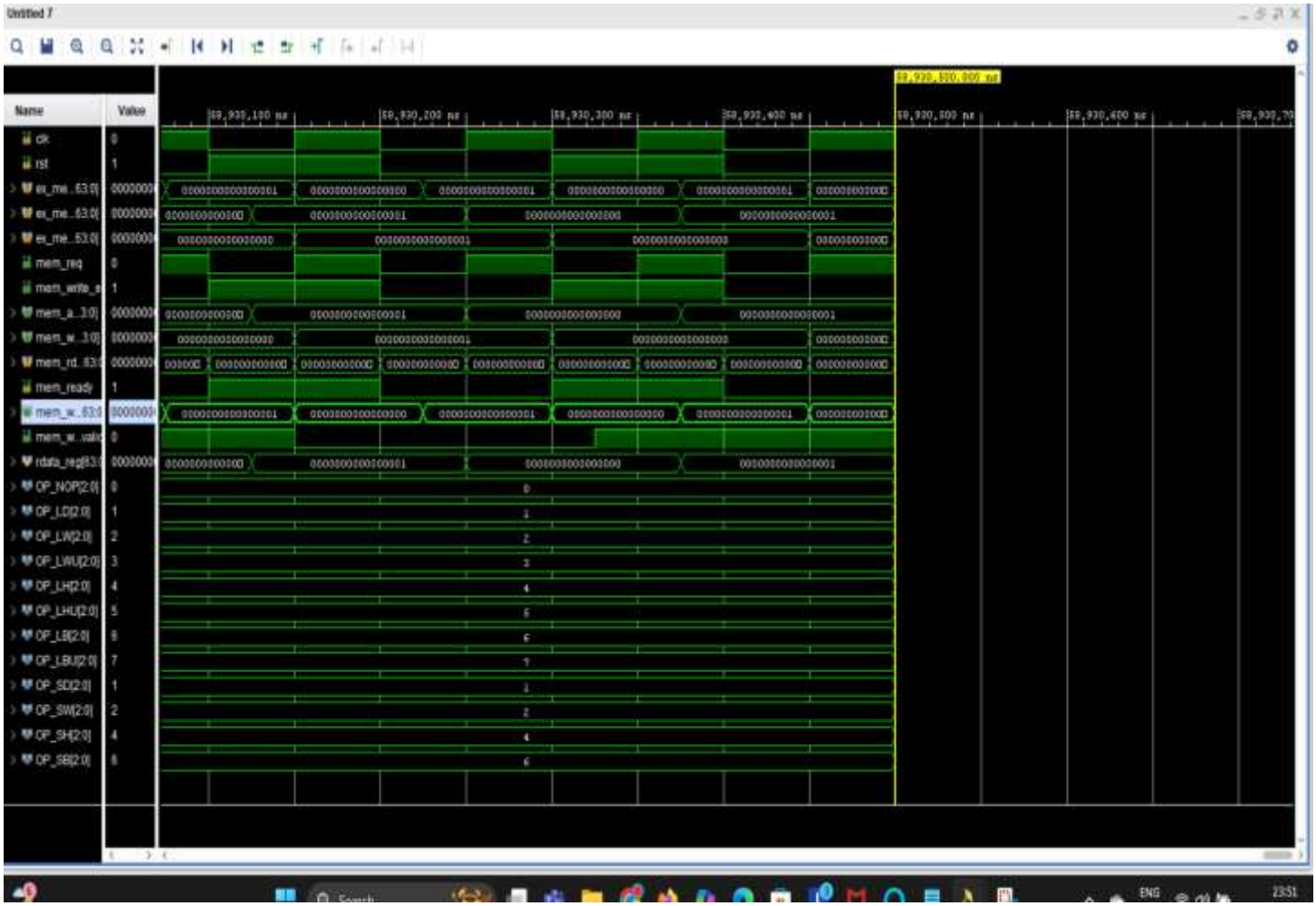

Fig.20: Memory Unit Output Waveform

The waveform confirms that:

- Load and store instructions are decoded properly.
- Data masking works (based on operation size).
- mem_req and mem_write_en toggle correctly.

The handshake (mem_ready) and mem_wb_valid pipeline signals are behaving as designed. So this simulation verifies your memory interface logic of a 64-bit RISC-V MEM stage.

Phase	What Happens	Key Signals
Reset	All cleared	mem_req=0, mem_write_en=0
Load (LD/LW/LH/LB)	Read memory → produce WB data	mem_req=1, mem_write_en=0, mem_wb_valid=1
Store (SD/SW/SH/SB)	Write memory → no WB data	mem_req=1, mem_write_en=1, mem_wb_valid=0
Memory Ready	Synchronization with mem_ready	Data captured only when ready=1

Table-9 Memory Unit Operation

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

The project **"Design and Implementation of Functional Units of 64-bit RISC-V Processor using Verilog"** successfully demonstrates the working of key processor components and their integration in a 64-bit architecture. The design and simulation of functional units such as the Arithmetic Logic Unit (ALU), Register File, and Control Unit have been implemented using Verilog HDL, ensuring modularity and scalability. Through simulation results and verification, the developed design proves to be efficient, accurate, and compliant with the RISC-V instruction set architecture. The project provides an in-depth understanding of processor design principles, hardware description language modeling, and the functional behavior of a modern open-source processor. Overall, this implementation forms a strong foundation for developing more complex RISC-V-based systems

Throughout the project, a thorough understanding of RISC-V architecture principles guided the design decisions, ensuring compliance with the base 32I instruction set while also incorporating pseudo-instructions to enhance programmability and readability. The inclusion of simultaneous read and write capabilities for registers and data memory further enriches the processor core's functionality and versatility. The evaluation of the processor core's performance, as evidenced by waveform analysis and functional testing, demonstrates its reliability and efficiency in executing instructions within the specified frequency of 2.2 KHz. However, future iterations could explore avenues for performance optimization, such as increasing memory capacity, incorporating advanced instruction extensions, or implementing pipeline stages to exploit instruction-level parallelism.

This work can be further extended to design a complete pipelined 64-bit RISC-V processor to improve instruction throughput and performance. Advanced features such as branch prediction, cache memory, hazard detection, forwarding units, and exception handling can be incorporated for real-time applications. The design can also be synthesized on an FPGA platform to validate hardware performance and power efficiency. Furthermore, the processor can be expanded to support floating-point operations, memory management units (MMU), and custom instruction extensions for AI and embedded system applications. With the rapid growth of open-source hardware, this project serves as a solid stepping stone toward the development of fully functional, high-performance RISC-V processors for academic research and industrial use. Another area of focus for future enhancements involves increasing the operating frequency of the processor core. By leveraging advancements in semiconductor technology and optimizing the design for higher clock speeds, the processor can achieve greater computational throughput and responsiveness, making it better suited for demanding computing tasks.

CHAPTER 9

REFERENCE

[1]. K. Diefendorff et al.(2000),'AltiVec extension to PowerPC accelerates media processing', IEEE Micro, Vol. 20, No. 2, pp. 85 –95.

[2] Anu Samanta et al (2024), ‘A Comprehensive Survey and Comparison on Pipelined RISC Systems Architectures’, J. Electrical Systems 20-3 (2024): 2817-2825

[3] Ravi Rajwar and James R. Goodman (2001),'Speculative lock elision: enabling highly concurrent multithreaded execution', IEEE Computer Society, pp. 294–305.

[4] M. Tremblay et al. (1996),'VIS speeds new media processing', IEEE Micro, Vol. 16, No. 4, pp. 10 –20.

[5] Marc Tremblay et al. (2000), 'The MAJC architecture: A synthesis of parallelism and scalability', IEEE Micro, Vol. 20, No. 6, pp. 12–25.

[6] A. Peleg and U. Weiser, "MMX technology extension to the Intel architecture," in IEEE Micro, vol. 16, no.4, pp. 42-50, Aug. 1996, doi: 10.1109/40.526924

[7] R.B. Lee (1996), 'Subword Parallelism with MAX-2’, IEEE Micro, pp. 51-59.

[8] Abbott, et al. (1996), 'Broadband Algorithms with the Micro Unity Media Processor', IEEE Computer Society, pp. 349-354.

[9] C. Hansen (1996), 'Micro Unity's Media Processor Architecture', IEEE Micro, pp. 34-41.

[10] Electrical Engineering and Computer Sciences University of California at Berkeley Technical Report No. UCB/EECS-2016-1

[11] Lincoln TX-2 computer, http://www.bitsavers.org/pdf/mit/tx-2/TX-2_Papers_WJCC_57.pdf

[12] https://pages.cs.wisc.edu/~markhill/restricted/ieeecomputer2005_arm.pdf

[13] https://riscv.org/technical/specifications/